Advanced Machine Learning

Tommaso Ferrario (@TommasoFerrario18)

Telemaco Terzi (@Tezze2001)

Ottobre 2024

Contents

1 Feed Forward Neural Network **5**

1.1 From Linear Models to Neural Networks . . . 5
1.1.1 The learning problem . . . 5
1.1.2 Models with a linear score . . . 5
1.1.3 The XOR problem . . . 6
1.1.4 Choosing the representation $\phi(x)$ . . . 6
1.1.5 Kernel trick . . . 6
1.2 The Feed Forward Network . . . 7
1.2.1 A network as a composition of functions . . . 7
1.2.2 Inside a layer: the forward pass . . . 7
1.2.3 Solving XOR: hidden units as feature detectors . . . 8
1.3 Hidden Units and Activation Functions . . . 8
1.3.1 Tanh . . . 9
1.3.2 Rectified Linear Unit . . . 9
1.3.3 Modern activation functions: GELU, SiLU and SwiGLU . . . 10
1.3.4 Guidelines for choosing an activation function . . . 11
1.4 Expressive Power and Architecture Design . . . 11
1.4.1 How one hidden layer builds shapes . . . 11
1.4.2 The Universal Approximation Theorem . . . 12
1.4.3 Why depth helps: composition and space folding . . . 12
1.4.4 Beyond the dense chain: connectivity and skip connections . . . 12
1.5 Cost Functions and Output Units . . . 13
1.5.1 Loss and cost . . . 13
1.5.2 Maximum likelihood: where the losses come from . . . 13
1.5.3 Regression losses . . . 13
1.5.4 Classification losses . . . 14
1.5.5 Output units . . . 14
1.6 Gradient-Based Optimization . . . 16
1.6.1 The objective and gradient descent . . . 16
1.6.2 Regularization . . . 16
1.6.3 Minibatch Stochastic Gradient Descent . . . 17
1.7 Backpropagation . . . 17
1.7.1 Chain rule . . . 17
1.7.2 The backward pass step by step . . . 17
1.8 Training in Practice . . . 18
1.8.1 Weight initialization . . . 18
1.8.2 The whole training procedure . . . 19
1.9 Regularization . . . 19
1.9.1 Regularization strategies . . . 19
1.10 Generative Adversarial Models . . . 23
1.11 Optimization for training deep models . . . 23
1.11.1 Batch and Minibatch . . . 24
1.11.2 Challenges in deep learning . . . 25
1.11.3 Optimization algorithms . . . 26

2 Autoencoder **29**
2.1 Types of Autoencoder . . . 29
2.1.1 Undercomplete Autoencoder . . . 29
2.1.2 Regularized Autoencoder . . . 30

3 Convolutional Neural Network **31**
3.1 CNNs components . . . 32
3.1.1 Linear Convolution . . . 32
3.1.2 Spatial pooling . . . 33
3.1.3 Local Response Normalization . . . 34
3.1.4 Input sensibility . . . 34
3.2 CNN Architectures . . . 34
3.2.1 Transfert learning . . . 35

4 Model Compression **37**
4.1 Weight Sharing . . . 37
4.2 Network pruning . . . 37
4.2.1 Categorization of Pruning Techniques . . . 38
4.2.2 Structured vs. Unstructured Pruning . . . 38
4.3 Low rank matrix and tensor decomposition . . . 39
4.4 Knowledge distillation . . . 39
4.5 Quantization . . . 40
4.6 Low resource and efficient architectures . . . 40

5 Recurrent Neural Networks **42**
5.1 Gated RNNs . . . 44
5.1.1 Long Short-Term Memory LSTM . . . 44
5.1.2 GRU . . . 46

6 Generative Adversarial Networks **47**
6.1 Generator architecture . . . 47
6.2 Generative Adversarial Networks . . . 48
6.2.1 Define the loss function . . . 48
6.3 Evaluating Generative Models . . . 50

7 Federated Learning **51**
7.1 Distributed Learning . . . 52
7.2 Baseline algorithm: FedAvg . . . 52

8 Transformers **54**
8.1 Dot-product self attention . . . 54
8.2 Positional encoding . . . 56
8.2.1 Absolute position embeddings . . . 57
8.2.2 Relative position embeddings . . . 57
8.3 Multiple heads self attention . . . 57
8.4 Transformer Layers . . . 57
8.5 NLP pipeline . . . 58

9 Self-Supervised Learning **60**
9.1 Learning visual features from pretexts tasks . . . 61
9.1.1 Generation-based pretext tasks . . . 61
9.1.2 Context-based pretext tasks . . . 62
9.1.3 Free semantic label-based pretext tasks . . . 63
9.1.4 Cross modal-based pretext tasks . . . 64
9.2 Commonly used downstream tasks for evaluation . . . 64

10 Introduction to Explainable AI **65**
10.1 Taxonomy of XAI Methods 65
10.1.1 Framework 65
10.1.2 Scope 65
10.1.3 Applicability 66
10.1.4 Produced Results 66
10.2 SHAP 66
10.3 LIME 66

Introduction

Deep Learning is a subset of machine learning that is concerned with neural networks that are use to learn underlying features in data.

In general, we can define machine learning as a program that starting from the input and the output of a system, learns the rules that govern the system. In order to obtain high performance, machine learning algorithms depends heavily on the **representation** of the data. Representation is therefore the fundamental, and many artificial intelligence tasks can be solved by designing the right set of features.

The most difference between Deep Learning and Machine Learning is that, the first one try to learn an efficient representation of data and than use it to train a learn model. The latter one use a representation of data specified by an expert to train a learn model.

Deep Learning use neural networks with many layers of activity vectors as representations and learning the connection strengths between that give rise to these vectors by following the stochastic gradient of an objective function that measures how well the network is performing.

So, the key ingredient of deep learning is **Depth**. There are two main ways to measure the depth of a model:

1. In terms of depth of the graph describing how concepts are related to each other;
2. In terms of number of sequential instructions that must be executed to evaluate the architecture. This can be influenced by the choice of basic functions used.

One solution to the problem of feature representation is to use machine learning not only to find the mapping between input and output, but also to find the representation itself. This approach is called **Representation Learning**. The goal of this task is to identify the *factor of variations* that explain the observed data.

Note 1. *The most common example of representation learning is the use of auto-encoders.*

A key part of representation learning consists in the **distributed representation**, which means a many to many relationship between two types of representation:

- Each concept is represented by many neurons.
- Each neuron participates in the representation of many concepts.

Deep learning solves this central problem in representation learning by introducing representations that are expressed in therms of other, simpler representations. An example is bunch of letters form words, sets of words form phrases.

Chapter 1

Feed Forward Neural Network

This chapter builds the feed forward network step by step. We start from the limits of linear models, which motivate the idea of learning a representation of the data (Section 1.1). We then describe the structure of the network and how it computes a prediction (Section 1.2), the hidden units that make it non-linear (Section 1.3) and what depth and width buy us in terms of expressive power (Section 1.4). The second half of the chapter is about learning: the loss functions and output units (Section 1.5), gradient-based optimization (Section 1.6), backpropagation (Section 1.7) and the practical details of training (Section 1.8).

1.1 From Linear Models to Neural Networks

1.1.1 The learning problem

Supervised learning is about building functions that analyze examples (instances) and generalize to unseen data. During training we learn parameters θ such that the model $f(x;\theta)$ approximates the underlying **target function** $f^*(x)$. Each training input $x \in \mathbb{R}^d$ comes with a label $y \sim f^*(x)$, and the model must produce a prediction $\hat{y}$ close to y.

Note 2. *We write $y \sim f^*(x)$ rather than $y = f^*(x)$ because real observations contain noise: we want a model that captures the underlying regularity and generalizes, not one that memorizes the noise.*

Searching among all possible functions is intractable, so we restrict the search to specific families of functions called **hypothesis classes**.

1.1.2 Models with a linear score

The simplest hypothesis class is made of **generalized linear models**. They compute a linear score $z = w^T x + b$ and pass it through a monotonic output function to obtain the prediction $\hat{y}$:

- the **identity**, $\hat{y} = z \in \mathbb{R}$, for linear regression;
- the **sign function**, for hard binary classification:

$$\hat{y} = \text{sign}(w^T x + b) \in \{-1, +1\} \tag{1.1}$$

- the **sigmoid**, for binary classification when we need calibrated probabilities (logistic regression):

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1) \tag{1.2}$$

- the **softmax**, for multi-class classification with k classes, which turns the k linear scores $z = W^T x + b$ into a probability vector:

$$\hat{y}_i = \text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}} \tag{1.3}$$

Note 3. ***Why these models always have flat decision boundaries.*** *Sigmoid and softmax are non-linear, but they only rescale the linear score z into $[0, 1]$. For binary classification the boundary $\hat{y} = 0.5$ corresponds to $z = w^T x + b = 0$, which is a hyperplane. With softmax, the boundary between class i and class j is the hyperplane $(w_i - w_j)^T x + (b_i - b_j) = 0$. Since no term combines the features (such as $x_1 x_2$), the effect of x_1 on the score is the same whatever the value of x_2.*

Linear models have one big advantage and one big limit.

- **Pro**: their loss landscape is convex, so they can be fit efficiently and reliably, with a single global minimum.
- **Con**: they can only separate classes whose boundary is a flat hyperplane, so they fail on any non-linearly separable pattern, unless the input space is transformed first.

1.1.3 The XOR problem

The classic example of this limit is the **XOR function**. Given two binary inputs $(x_1, x_2) \in \{0,1\}^2$, XOR outputs $y = 1$ when the inputs differ, $(0,1)$ and $(1,0)$, and $y = 0$ when they are equal, $(0,0)$ and $(1,1)$.

No straight line separates the two classes. To see it, suppose a linear score $z = w_1x_1 + w_2x_2 + b$ were negative at $(0,0)$ and $(1,1)$ and positive at $(0,1)$ and $(1,0)$:

- at $(0,0)$ we need $b < 0$;
- at $(1,0)$ and $(0,1)$ we need $w_1 + b > 0$ and $w_2 + b > 0$; summing the two gives $w_1 + w_2 + 2b > 0$, hence $w_1 + w_2 + b > -b > 0$;
- but at $(1,1)$ we need $w_1 + w_2 + b < 0$, a direct contradiction.

In practice, a linear regression with mean squared error on the four XOR points compromises by setting $w_1 = w_2 = 0$ and $b = 0.5$, predicting $\hat{y} = 0.5$ everywhere.

1.1.4 Choosing the representation $\phi(x)$

If the target $f^*(x)$ is **non-linear**, a linear model applied to the raw input cannot approximate it. The solution is to apply a **non-linear transformation** $\phi(x)$ to the input, mapping each instance into a new feature vector in which the problem becomes linear. There are three ways to obtain ϕ:

- **Fixed generic transformations** (kernels, polynomials). They are simple, but generic expansions, such as adding all the products x_ix_j, explode combinatorially in high dimension and do not adapt to the structure of the task.
- **Manually engineered features**. A domain expert designs ϕ for one application. This requires a lot of human effort and does not transfer to other domains.
- **Learned representations**, i.e. deep learning. We parameterize the feature map as $\phi(x;\theta)$ and learn θ from the data, together with the parameters ω of the final linear predictor:

$$f(x;\theta,\omega) = \phi(x;\theta)^T \cdot \omega \tag{1.4}$$

Note 4. *In a deep learning model $\phi(x;\theta)$ corresponds to the hidden layers: they learn to deform the input space, so that the output layer only has to apply a linear model.*

1.1.5 Kernel trick

A fixed transformation into a higher-dimensional space can make non-linearly separable data linearly separable, but computing $\phi(x)$ explicitly in a very high (or infinite) dimension can be prohibitive. **Kernels** avoid the problem.

Note 5. *Many transformations make the data linearly separable in higher dimension, but only those that correspond to an inner product in some feature space are valid kernels.*

The **kernel trick** lets us work in the high-dimensional feature space without ever computing the coordinates $\phi(x)$. If we write the weight vector as a linear combination of the m training instances, a linear model becomes

$$\omega^T x + b = b + \sum_{i=1}^{m} \alpha_i x^T x^{(i)} \tag{1.5}$$

where b is the bias, α_i a learned coefficient and $x^{(i)}$ the i-th training instance. Replacing x with $\phi(x)$ gives

$$f(x) = b + \sum_{i=1}^{m} \alpha_i \phi(x)^T \phi(x^{(i)}) = b + \sum_{i=1}^{m} \alpha_i k(x, x^{(i)}) \tag{1.6}$$

The kernel function $k(x, x^{(i)}) = \phi(x)^T \phi(x^{(i)})$ returns the inner product of the transformed points by operating directly on the original inputs. Since ϕ is fixed, optimizing the coefficients α_i is still a convex problem. In Support Vector Machines (SVMs) only a sparse subset of the training points, the **support vectors**, have $\alpha_i \neq 0$. Any algorithm built on this principle is called a **kernel machine** or **kernel method**.

1.2 The Feed Forward Network

1.2.1 A network as a composition of functions

A **Feed Forward Neural Network** (FFNN), also called **Multilayer Perceptron** (MLP), overcomes the limits of linear models without hand-crafted features. It arranges layers of units in a chain in which every layer feeds the next one, with no feedback loops, and each layer can be seen as a function from vectors to vectors.

A FFNN is therefore a composition of functions, one per layer. A network with three layers that approximates $f(x)$ has the form

$$f(x) = f^{(3)}(f^{(2)}(f^{(1)}(x)))$$

where $f^{(i)}$ is the function computed by the i-th layer. The number of layers in the chain is the **depth** of the model, while the number of units of the hidden layers is its **width**.

The training data only say what the output layer must produce at each point x; they do not tell the intermediate layers what to compute. These are called **hidden layers** precisely because the learning algorithm has to decide by itself how to use them to build a useful internal representation of the input.

1.2.2 Inside a layer: the forward pass

The **forward pass** takes the input $h^{(0)} = x$ through the L layers up to the prediction $\hat{y}$. In each layer $l \in \{1, \dots, L\}$ a set of units receives the activations $h^{(l-1)}$ of the previous layer and performs two steps.

1. The **pre-activation** is an affine transformation:

$$z^{(l)} = W^{(l)T} h^{(l-1)} + b^{(l)} \tag{1.7}$$

   where $W^{(l)} \in \mathbb{R}^{d_{l-1} \times d_l}$ has one row per input and one column per unit of layer l, and $b^{(l)} \in \mathbb{R}^{d_l}$ is the bias vector. In the last layer, the pre-activation $z = z^{(L)}$ is called the **logit** (or linear score).

2. The **activation** applies a non-linear function g component-wise:

$$h^{(l)} = g\left(z^{(l)}\right) \tag{1.8}$$

   In the output layer, this role is played by the output function (e.g. the sigmoid), which turns the logit into the prediction $\hat{y}$.

Note 6. ***Counting the parameters.*** *A layer that maps d_{l-1} inputs to d_l units has $d_{l-1} \cdot d_l$ weights and d_l biases, i.e.*

$$Parameters_l = (d_{l-1} + 1) \cdot d_l$$

and the whole network has $\sum_{l=1}^{L} (d_{l-1} + 1)\, d_l$ parameters. When the input dimension is large, most of them are typically in the first layer.

Note 7. ***Without non-linearity, depth adds nothing.*** *If g were linear ($g(z) = z$), stacking layers would collapse into a single linear transformation:*

$$W_2^T (W_1^T x + b_1) + b_2 = \underbrace{(W_1 W_2)^T}_{W'^T} x + \underbrace{(W_2^T b_1 + b_2)}_{b'}$$

A deep linear network represents exactly the same functions as a single linear layer. Non-linear activations between the layers are what make the composition more expressive than its parts.

1.2.3 Solving XOR: hidden units as feature detectors

A network with **one hidden layer of two ReLU units**, $g(z) = \max(0, z)$, solves XOR by changing the representation of the data:

$$h = \max\left(0, \begin{bmatrix}1 & 1\\ 1 & 1\end{bmatrix}\begin{bmatrix}x_1\\ x_2\end{bmatrix} + \begin{bmatrix}0\\ -1\end{bmatrix}\right), \qquad \hat{y} = \begin{bmatrix}1 & -2\end{bmatrix} h$$

The first hidden unit, $h_1 = \max(0, x_1 + x_2)$, is active when at least one input is 1 (it acts as an OR detector). The second, $h_2 = \max(0, x_1 + x_2 - 1)$, is active only when both inputs are 1 (an AND detector). The output $\hat{y} = h_1 - 2h_2$ is then $0, 1, 1, 0$ on the four XOR points.

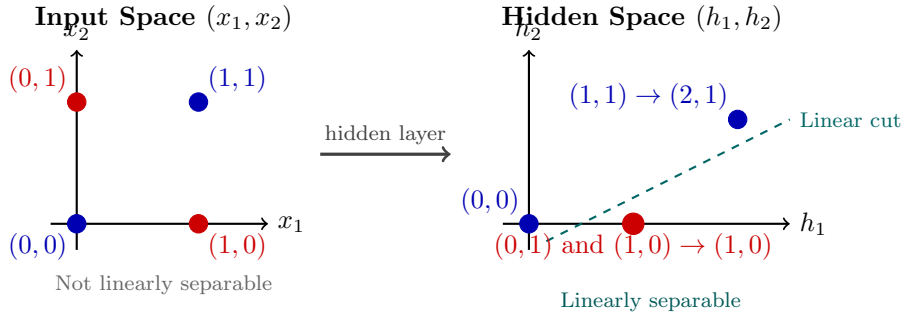


Figure 1.1: The hidden layer maps both positive points $(0, 1)$ and $(1, 0)$ onto the same point $(1, 0)$ of the hidden space, where a single linear cut separates the two classes.

As Figure 1.1 shows, the hidden layer does not bend the output classifier: it *folds* the input space so that $(0, 1)$ and $(1, 0)$ land on the same point $h = (1, 0)$, and in the new representation a simple linear cut is enough.

Note 8. ***Why width helps optimization.*** *Two ReLU units are enough to represent XOR, but training such a network from a random initialization often fails: if a unit starts with $z < 0$ on all four points, its gradient is zero everywhere and the unit stays dead. With a slightly wider layer (say 8 or 16 units) gradient descent has many redundant candidates, and nearly every random initialization reaches 100% accuracy.*

More generally, every hidden unit acts as a **feature detector**. The score $z = w^T x + b = \|w\|\|x\| \cos\gamma + b$, where γ is the angle between w and x, is largest, for a given norm of x, when x is aligned with w. The unit therefore works as a template matcher: it responds strongly when a specific pattern (a stroke, an edge) is present in the input, and stays quiet otherwise. In a deep network the layers compose these features hierarchically: the first layer detects simple local patterns, the second combines them into parts or corners, and the deeper ones assemble abstract concepts.

1.3 Hidden Units and Activation Functions

Every hidden unit computes a pre-activation z and passes it through an **activation function** $h = g(z)$. A good hidden activation must meet three requirements:

1. it must be **non-linear**, so that stacking layers adds expressive power;
2. its derivative $g'(z)$ must stay **large enough** where z typically falls, so that gradients can flow backward through many layers;
3. it must be **cheap to evaluate**, since $g(z)$ and $g'(z)$ are computed millions of times during training.

Saturation and the vanishing gradient problem A unit **saturates** when $|z|$ is so large that it falls in a flat region of g, where $g'(z) \approx 0$. Backpropagation multiplies the error signal by $g'(z)$, so a saturated unit receives almost no gradient and stops learning.

This is why classical sigmoids are poor hidden activations in deep networks. Consider a chain of $L = 10$ hidden layers: by the chain rule the gradient of the loss with respect to a weight w_1 of the first layer contains the product of the slopes of all the activations,

$$\frac{\partial \mathcal{L}}{\partial w_1} \propto \prod_{l=1}^{L} g'(z_l)$$

The slope of the logistic sigmoid, $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, is at most $\frac{1}{4}$ (at $z = 0$) and drops below 0.05 for $|z| > 3$. Even in the best case, the product of ten such slopes is at most $(1/4)^{10} \approx 10^{-6}$. The output layer receives a normal gradient and learns quickly, while the first layers receive a signal a million times weaker and barely move. This exponential decay of the backward signal is the **vanishing gradient problem**.

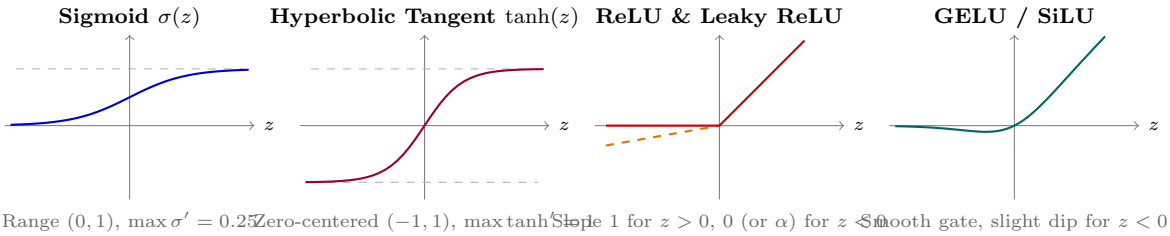


Figure 1.2: The main hidden activation functions: saturating sigmoids (σ and tanh), piecewise linear rectifiers (ReLU and Leaky ReLU) and smooth self-gated units (GELU/SiLU).

1.3.1 Tanh

The **hyperbolic tangent** is a rescaled and shifted logistic sigmoid:

$$\tanh(z) = 2\sigma(2z) - 1 \in (-1, 1) \tag{1.9}$$

When a bounded, sigmoid-like activation is needed in hidden layers (or in recurrent networks), tanh is preferred over the logistic sigmoid for two reasons.

1. **Larger slope near zero**: $\tanh'(0) = 1$ against $\sigma'(0) = 0.25$, so gradients survive through more layers while the pre-activations stay small.

2. **Zero-centered outputs**: the logistic sigmoid only outputs positive numbers ($h_j > 0$). Since the gradient of a weight of the next layer is $\frac{\partial \mathcal{L}}{\partial w_j} = \delta \cdot h_j$, all the weights entering the same neuron get gradients of the **same sign** at every step, and gradient descent zig-zags toward the minimum. Since $\tanh(0) = 0$, its outputs take both signs, avoiding the problem.

Note 9. *The logistic sigmoid $\sigma(z) \in (0, 1)$ belongs to the **output layer** of binary classifiers, where it predicts a probability, while $\tanh(z) \in (-1, 1)$ is preferred when a saturating activation is needed in the **hidden layers**. Even* tanh *saturates for $|z| > 3$, since $\tanh'(z) = 1 - \tanh^2(z) \to 0$.*

1.3.2 Rectified Linear Unit

The **Rectified Linear Unit** (ReLU) is the default activation of most feedforward and convolutional networks:

$$g(z) = \max(0, z), \qquad g'(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z < 0 \end{cases} \tag{1.10}$$

ReLU greatly alleviates the vanishing gradient problem: while a unit is active its slope is exactly 1, so it never saturates for positive inputs and the gradient can cross dozens of layers without shrinking. It is also extremely cheap (a threshold at zero) and produces **sparse representations**, since units with negative pre-activation are switched off. The function is not differentiable at $z = 0$, but in practice optimization simply uses the left or right derivative (usually $g'(0) = 0$) without any problem.

Dead units: the price of a zero slope The main drawback of ReLU is the **dying ReLU problem**. If the weights and bias of a unit end up in a region where $z < 0$ for **every** training example, the unit outputs 0 and has derivative 0 everywhere: it receives no gradient on any batch, its weights never change again, and the neuron is permanently dead. This usually comes from a bad initialization (e.g. a large negative bias) or from a **learning rate that is too large**, whose single big step pushes the weights deep into the negative half-space. To prevent dead units we can lower the learning rate, initialize the biases to a small positive constant (e.g. 0.1), or use a generalization of ReLU that keeps a non-zero slope for $z < 0$.

ReLU generalizations Most piecewise linear variants of ReLU come from one formula:

$$h_i = g(z_i, \alpha_i) = \max(0, z_i) + \alpha_i \min(0, z_i) \tag{1.11}$$

and differ only in how α_i is chosen:

- **Absolute value rectification**: $\alpha_i = -1$, so $g(z) = |z|$. It is used in the early stages of object recognition, where features should not change when the polarity is reversed (dark-on-light versus light-on-dark edges).
- **Leaky ReLU**: α_i is a small positive constant such as 0.01. The small slope for negative inputs lets even units with $z < 0$ receive a gradient and recover, which eliminates dead units.
- **Parametric ReLU** (PReLU): α_i is learned by backpropagation together with the weights, so each channel or layer can adapt its negative slope to the data.
- **Maxout units**: instead of a fixed activation, a maxout unit computes k linear filters and returns the maximum:

  $$g(x) = \max_{j \in \{1,\dots,k\}} \left(w_j^T x + b_j \right) \tag{1.12}$$

  The unit learns its own activation as a convex, k-piece linear function (Figure 1.3), which generalizes ReLU, Leaky ReLU and $|z|$. The maximum always routes the gradient through the active piece, so maxout does not saturate and has no dead units, at the cost of k times more parameters per unit.

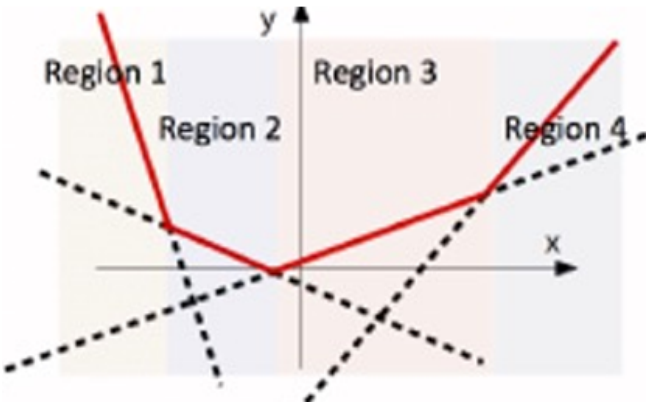


Figure 1.3: Maxout units

- **Softplus**: a smooth approximation of ReLU, whose derivative is the logistic sigmoid ($\zeta'(z) = \sigma(z)$):

  $$\zeta(z) = \log(1 + e^z) \tag{1.13}$$

  It was introduced to remove the kink of ReLU at $z = 0$, but experiments consistently show that plain ReLU trains faster and generalizes better in hidden layers. Softplus is mostly useful in output layers, to force a positive prediction (for example a variance).

1.3.3 Modern activation functions: GELU, SiLU and SwiGLU

Modern architectures, especially Transformers and large language models, replace the sharp kink of ReLU with smooth **gated** activations. The idea is to see ReLU as the product of z and a hard binary gate $\mathbb{I}(z > 0) \in \{0, 1\}$, and to replace the hard gate with a smooth one.

- **GELU** (Gaussian Error Linear Unit) is the standard hidden activation of BERT, GPT and Vision Transformers. Its gate is $\Phi(z) = P(Z \leq z)$, the cumulative distribution function of a standard normal variable $Z \sim \mathcal{N}(0, 1)$:

  $$\text{GELU}(z) = z \cdot \Phi(z) \approx z \cdot \sigma(1.702z) \tag{1.14}$$

  Large positive values pass almost unchanged ($\Phi \to 1$), large negative values are blocked ($\Phi \to 0$), and small negative values are partially let through. The curve is smooth, with a slight dip below zero (down to -0.17 near $z \approx -0.75$) that prevents dead units.

- **SiLU** (Sigmoid Linear Unit, or Swish) applies the same principle using the logistic sigmoid as the gate; it is used in convolutional models such as EfficientNet:

$$\mathrm{SiLU}(z) = z \cdot \sigma(z) \tag{1.15}$$

- **SwiGLU** (Swish-Gated Linear Unit) is used in the feedforward blocks of large language models such as LLaMA and PaLM. GELU and SiLU use z to gate itself, while SwiGLU **learns a separate gate** from the input x, with two parallel projections W_a (content) and W_b (gate):

$$h = (W_a^T x + b_a) \odot \mathrm{SiLU}(W_b^T x + b_b) \tag{1.16}$$

  where $\odot$ is the element-wise product. A SwiGLU block has three weight matrices (two projections in, one out) instead of two, so the hidden width is usually reduced to $\frac{2}{3}$ of a standard MLP layer to keep the number of parameters and FLOPs roughly the same.

1.3.4 Guidelines for choosing an activation function

In practice a few rules of thumb are enough:

- **Hidden layers of MLPs and CNNs**: start with **ReLU** and He initialization (Section 1.8). It is fast, simple and has slope 1 for active units.
- **Many dead units**: switch to **Leaky ReLU** (or PReLU) and lower the learning rate.
- **Transformer feedforward blocks**: use **GELU** or **SwiGLU**, which have smooth gradients and perform better at scale.
- **Recurrent networks** (RNNs, LSTMs): use the **sigmoid** for the gates, which must lie in $[0, 1]$, and **tanh** for the state updates, which must be bounded and zero-centered in $[-1, 1]$.
- **Output layer**: never use ReLU at the output of a classifier; match the output unit to the target distribution (linear for regression, sigmoid for binary classification, softmax for multi-class classification).

1.4 Expressive Power and Architecture Design

Designing a feedforward network means choosing its **width** (units per layer) and its **depth** (number of layers). To understand why modern networks prefer depth to sheer width, we first see how a single hidden layer builds functions.

1.4.1 How one hidden layer builds shapes

In one dimension, a sigmoid unit $\sigma(w(x-c))$ is a soft step that turns on at $x = c$. The difference of two shifted sigmoids, $\sigma(w(x-a)) - \sigma(w(x-b))$, steps up at a and down at b, producing a localized **bump**. Likewise, three ReLU units with output weights $(+1, -2, +1)$ build a triangular **hat** (Figure 1.4, left): the first ReLU starts a rising ramp, the second bends it downward at the peak, and the third flattens it back to zero. Summing many bumps or hats placed at different locations and scaled by the output weights, a single hidden layer can trace any continuous curve as a piecewise linear approximation.

In two or more dimensions each hidden unit defines a **cut** of the input space along the hyperplane $w^T x + b = 0$, with high values on one side and low values on the other. The hidden layer places the cuts and the output layer combines them like soft logical operators:

- the difference of two parallel cuts isolates a **band**;
- the sum of two crossing cuts, with a high bias threshold, acts as a logical AND and carves out a **corner**;
- the difference of two crossing cuts acts as an AND NOT and isolates a **wedge**.

1.4.2 The Universal Approximation Theorem

This intuition is formalized by the **Universal Approximation Theorem** (Hornik et al., 1989; Cybenko, 1989; Leshno et al., 1993). For any continuous function f^* on a closed and bounded subset of $\mathbb{R}^d$ and any tolerance $\varepsilon > 0$, there is a network with **one hidden layer** of N units and a linear output layer,

$$f(x) = \sum_{j=1}^{N} v_j \, g\left(w_j^T x + b_j\right) + c \tag{1.17}$$

such that $|f(x) - f^*(x)| < \varepsilon$ on the whole domain, as long as the activation g is continuous and non-polynomial (true for sigmoid, tanh and ReLU).

Note 10. ***What the theorem does not say.*** *It guarantees that a single hidden layer* can *represent any continuous function, but with three caveats:*

1. ***the width can be exponential****: in d dimensions, placing local bumps or kinks on a grid to match an arbitrary function may require $N \sim \mathcal{O}(\varepsilon^{-d})$ units;*
2. ***no optimization guarantee****: the right weights exist, but gradient descent is not guaranteed to find them;*
3. ***no generalization guarantee****: a huge single layer may simply memorize the training points and generalize poorly.*

1.4.3 Why depth helps: composition and space folding

Adding units to one layer **adds** linear pieces (kinks), while stacking layers **composes** functions and **multiplies** the pieces.

Take a symmetric "tent" function $t(x)$ on $[0, 1]$ built with 2 ReLU units: it rises from 0 to 1 at $x = 0.5$ and falls back to 0 at $x = 1$, so it has 2 linear pieces. If its output is fed into an identical tent layer, the composition $t(t(x))$ folds the interval in half before applying the tent again, and has $2 \times 2 = 4$ pieces. Stacking L layers uses only $2L$ units but produces 2^L linear pieces (Figure 1.4, right), a function that a single layer would need about 2^L units to build. More generally (Montúfar et al., 2014), the number of linear regions of a ReLU network grows **polynomially with the width** but **exponentially with the depth**.

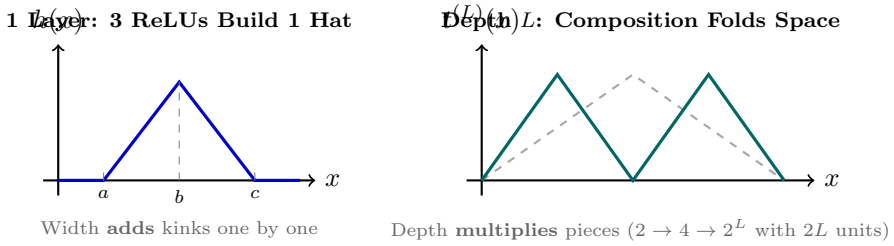


Figure 1.4: Left: three ReLU units of a single layer combine into a localized "hat". Right: composing a 2-ReLU "tent" over L layers folds the input space again and again, doubling the number of linear segments at each layer.

Geometrically, each layer with a folding activation (a tent, or $|z|$) folds the input space like a sheet of paper along a hyperplane, so whatever the following layers learn is automatically replicated on both halves of the fold (2^L copies after L layers). Choosing a deep architecture therefore encodes a strong prior: the target function is a **hierarchical composition of simpler steps**. This is why deep networks generalize much better on real tasks than shallow ones with the same number of parameters.

1.4.4 Beyond the dense chain: connectivity and skip connections

A standard MLP connects every unit of layer $l-1$ to every unit of layer l, without assumptions on the structure of the input. When the data have known symmetries we specialize the connectivity: **convolutional networks** (CNNs) use local connections and share weights across space for images, while **recurrent networks** (RNNs) share weights across time for sequences.

Finally, with very deep networks (30 to 100+ layers) training can degrade even with ReLU and a good initialization. Modern architectures solve this with **skip (residual) connections**:

$$h^{(l+1)} = h^{(l)} + F(h^{(l)}) \tag{1.18}$$

Instead of re-learning the whole representation, the layer only learns an additive correction $F(h^{(l)})$ to its input. In backpropagation the identity path $+h^{(l)}$ is a direct highway along which gradients flow across many layers unimpeded.

1.5 Cost Functions and Output Units

Now that we know how the network computes a prediction, we need to measure how good it is, and to make sure that its last layer produces something the loss can evaluate. The two choices are tightly coupled: the loss comes from the probability distribution we assume for the target, and the output unit produces the parameters of that distribution.

1.5.1 Loss and cost

Definition 1 (**Loss function**). *A **loss function** $\mathcal{L}(\hat{y}, y)$ maps the prediction of the model on a single data point and its true target to a real number that quantifies the error on that example.*

Definition 2 (**Cost function**). *A **cost function** $J(\theta)$ is the average loss over the whole training set (often combined with a regularization term). It is the overall objective that the learning algorithm seeks to minimize.*

1.5.2 Maximum likelihood: where the losses come from

Rather than picking a loss arbitrarily, deep learning derives it from one principle, **Maximum Likelihood Estimation** (MLE). Instead of seeing the network as outputting a bare number, we see it as predicting the parameters of a conditional distribution $p_{\text{model}}(y \mid x; \theta)$ of the target y given the input x.

MLE looks for the parameters that give the highest probability to the observed targets. Maximizing the product of the probabilities of N independent examples is equivalent to minimizing the average **Negative Log-Likelihood** (NLL), which is also the **cross-entropy** between the empirical data distribution and the model distribution:

$$J(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log p_{\text{model}}(y_i \mid x_i; \theta) \tag{1.19}$$

Every standard loss for regression and classification follows once we choose the family of p_{model}.

1.5.3 Regression losses

- **Mean Squared Error** (MSE) comes from MLE by assuming that the target is the network output plus Gaussian noise, $p_{\text{model}}(y \mid x) = \mathcal{N}(y; \hat{y}(x), \sigma^2)$:

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2 \tag{1.20}$$

  Squaring means that doubling the error quadruples the penalty, so large outliers dominate the gradient. The optimal predictor of the MSE is the **conditional mean** $\hat{y}^*(x) = \mathbb{E}[y \mid x]$.

- **Mean Absolute Error** (MAE) comes from assuming a heavy-tailed **Laplace** distribution around $\hat{y}(x)$:

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i| \tag{1.21}$$

  The gradient with respect to $\hat{y}_i$ is ± 1 whatever the size of the error, so large errors are not amplified and the loss is robust to outliers. The optimal predictor is the **conditional median** $\hat{y}^*(x) = \text{median}(y \mid x)$. The price is a derivative that is discontinuous at $y_i = \hat{y}_i$.

- **Huber loss** combines the best of the two to handle outliers smoothly: quadratic for small errors, which gives a smooth gradient near zero, and linear for large errors, which keeps outliers from hijacking the training:

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N} \begin{cases} \frac{1}{2}(y_i - \hat{y}_i)^2 & \text{if } |y_i - \hat{y}_i| \leq \delta \\ \delta|y_i - \hat{y}_i| - \frac{1}{2}\delta^2 & \text{otherwise} \end{cases} \tag{1.22}$$

1.5.4 Classification losses

- **Binary cross-entropy** (log-loss) comes from a **Bernoulli** distribution with $y_i \in \{0, 1\}$ and predicted probability $\hat{y}_i = P(y_i = 1 \mid x_i)$:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N} \Big(y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \Big) \tag{1.23}$$

  If the model is confident and right ($\hat{y}_i \to y_i$) the loss goes to 0; if it is confident and wrong ($\hat{y}_i \to 1 - y_i$) the loss blows up to infinity, so overconfident mistakes are punished heavily.

- **Categorical cross-entropy** comes from a **Categorical** distribution over k mutually exclusive classes. If $y_{i,c} \in \{0, 1\}$ is the one-hot indicator of the true class of example i:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{k} y_{i,c} \log(\hat{y}_{i,c}) = -\frac{1}{N}\sum_{i=1}^{N} \log(\hat{y}_{i,y_i}) \tag{1.24}$$

  Only the true class has a non-zero indicator, so the loss of each example depends only on the probability $\hat{y}_{i,y_i}$ assigned to the correct class.

- **Hinge loss** is a margin-based alternative for binary classification, with labels $y_i \in \{-1, +1\}$ and z_i the raw linear score:

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N} \max(0, 1 - y_i z_i) \tag{1.25}$$

  The product $y_i z_i$ is the **margin**, positive when the sign of the prediction is correct. Cross-entropy never reaches zero and keeps pushing the logits to grow, whereas the hinge loss is exactly 0 when $y_i z_i \geq 1$: once an example is correctly classified beyond the safety margin, it stops contributing to the gradient.

Note 11. *Any custom loss can be used in deep learning, as long as it is differentiable almost everywhere, so that it stays compatible with gradient-based optimization.*

1.5.5 Output units

The hidden layers extract a feature vector h. The **output unit** is the last layer: it transforms the logits $z = W^T h + b$ into the parameters of the conditional distribution $p_{\text{model}}(y \mid x)$ chosen above.

Linear output unit

The linear unit returns the affine transformation directly:

$$\hat{y} = W^T h + b \tag{1.26}$$

It is used in regression, where $\hat{y} \in \mathbb{R}$ is the mean of a conditional Gaussian $\mathcal{N}(y; \hat{y}, \sigma^2)$, and maximum likelihood reduces to minimizing the MSE. Its great advantage is that its slope is always 1: it **never saturates**, so the gradient $\hat{y} - y$ reaches the network without shrinking, however far the prediction is from the target.

If we also want the network to predict its own, input-dependent uncertainty (*heteroscedastic* regression), the output layer emits two values: the mean $\hat{y}(x)$ and a raw value $v(x)$, which goes through a softplus, $s^2(x) = \log(1 + e^{v(x)})$, so that the predicted variance is always strictly positive.

Sigmoid output unit

The sigmoid unit is used for **binary classification** ($y \in \{0, 1\}$) and models a Bernoulli distribution. From one scalar logit $z = w^T h + b$ it produces a valid probability $\hat{y} = \sigma(z) \in (0, 1)$. The logit is the **log-odds** of the positive class, since $\log \frac{\hat{y}}{1-\hat{y}} = z$: we have $\hat{y} = 0.5$ when $z = 0$, $\hat{y} > 0.5$ when $z > 0$ and $\hat{y} < 0.5$ when $z < 0$.

Note 12. ***Why a sigmoid output is paired with cross-entropy and not with MSE.*** *Suppose the true label is $y = 1$, but the model is confidently wrong, with $z = -5$ ($\hat{y} \approx 0.007$). The sigmoid is deep in its flat region, where $\sigma'(z) = \hat{y}(1 - \hat{y}) \approx 0.007$.*

- *With the **squared error** $\mathcal{L} = (\hat{y} - y)^2$ the chain rule multiplies the error by $\sigma'(z)$, and the gradient is tiny: the network barely learns exactly when it is most wrong.*
- *With the **binary cross-entropy** $\mathcal{L} = -y \log \hat{y} - (1-y) \log(1-\hat{y}) = \log(1 + e^{(1-2y)z})$ the logarithm undoes the exponential of the sigmoid, and the gradient with respect to the logit simplifies to*

$$\frac{\partial \mathcal{L}}{\partial z} = \hat{y} - y$$

With $y = 1$ and $\hat{y} \approx 0$ the gradient is -1: strong, and it never saturates when the model is wrong. It flattens out only as $\hat{y} \to y$, when the model is already right.

For **multi-label classification**, where an input can belong to several non-exclusive categories at the same time (e.g. an image tagged "outdoor" and "dog"), we use k independent sigmoid units trained with the sum of k binary cross-entropies.

Softmax output unit

The softmax unit generalizes the sigmoid to **multi-class classification** with k mutually exclusive classes, modeling a Categorical distribution. A linear layer produces k un-normalized logits $z \in \mathbb{R}^k$, and the softmax exponentiates and normalizes them, so that they are positive and sum to 1.

Combined with categorical cross-entropy, the negative log-likelihood of an example of true class y is

$$\mathcal{L}(z, y) = -\log \operatorname{softmax}(z)_y = -z_y + \log \sum_{j=1}^{k} e^{z_j} \tag{1.27}$$

which shows how the learning works. The first term, $-z_y$, pushes the logit of the correct class **up**. The second, $\log \sum_j e^{z_j} \approx \max_j z_j$, pushes all the logits **down**, in proportion to their predicted probability $\hat{y}_j$ (hardest on the strongest competitor). As in the binary case, the log cancels the exp and the gradient is the clean vector $\nabla_z \mathcal{L} = \operatorname{softmax}(z) - e_y$, which does not vanish when the prediction is wrong.

Shift invariance and numerical stability Adding the same constant c to all the logits does not change the probabilities:

$$\operatorname{softmax}(z + c)_i = \frac{e^{z_i + c}}{\sum_j e^{z_j + c}} = \frac{e^c e^{z_i}}{e^c \sum_j e^{z_j}} = \operatorname{softmax}(z)_i$$

On a computer, e^{z_i} **overflows** to ∞ if z_i is large and positive (e.g. 1000), and all the exponentials **underflow** to 0 (giving 0/0) if all the z_i are large and negative. The standard solution is to subtract $c = \max_j z_j$ before exponentiating:

$$\operatorname{softmax}(z) = \operatorname{softmax}\big(z - \max_j z_j\big) \tag{1.28}$$

The largest exponent is then $e^0 = 1$ and all the others are in $[0, 1]$, so there is neither overflow nor division by zero.

Gaussian Mixtures output unit

In some regression problems, especially inverse ones such as finding the joint angles of a robot arm that reach a given hand position, the same input x admits **several valid answers** y: the problem is **multimodal**. A single linear output trained with MSE is the wrong tool: it predicts the conditional mean $\mathbb{E}[y \mid x]$, which averages the valid branches and lands between them, possibly on an answer that is not valid at all.

To capture several peaks for the same x, a **Mixture Density Network** (MDN) uses a **Gaussian Mixture Model** (GMM) as output distribution:

$$p(y \mid x) = \sum_{i=1}^{K} \pi_i(x)\, \mathcal{N}\big(y; \mu^{(i)}(x), \Sigma^{(i)}(x)\big) \tag{1.29}$$

It is trained by minimizing the negative log-likelihood $-\log p(y \mid x)$. The last layer emits three groups of outputs, each passed through a transformation that enforces its constraint:

- the **mixture weights** $\pi_i(x) = p(c = i \mid x)$, the probability of selecting the i-th Gaussian. They must be positive and sum to 1, so they use a **softmax**;
- the **means** $\mu^{(i)}(x)$, the center of each branch. They are unconstrained, so they use **linear** units;
- the **variances/covariances** $\Sigma^{(i)}(x)$, the spread around each branch. They must be strictly positive, so the raw outputs go through a **softplus** ($\log(1 + e^v)$) or an exponential (e^v).

1.6 Gradient-Based Optimization

1.6.1 The objective and gradient descent

Over the n training samples, the objective (cost) function that measures the discrepancy between the learned function and the true targets is

$$J(\theta) = \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}(x_i, y_i, \theta) \tag{1.30}$$

and training looks for the parameters that minimize it:

$$\hat{\theta} = \arg\min_{\theta} J(\theta) = \arg\min_{\theta} \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}(x_i, y_i, \theta) \tag{1.31}$$

To minimize J we compute its **gradient** $\nabla_\theta J(\theta)$, the vector whose i-th component is the partial derivative $\frac{\partial J}{\partial \theta_i}$, i.e. how the cost changes when only θ_i is varied slightly. The gradient points in the direction of steepest increase, so **gradient descent** moves in the opposite direction:

$$\theta' = \theta - \eta \nabla_\theta J(\theta) \tag{1.32}$$

where $\eta > 0$ is the **learning rate**, which controls the size of the step. Gradient descent has converged when every component of the gradient is zero.

The best learning rate may change during training. A practical strategy is to start with a moderately large η, which makes fast progress at the beginning and helps to leave shallow regions, and then to decrease it gradually, so that the parameters can settle near a minimum.

Note 13. ***Convex and non-convex landscapes.*** *For linear models $J(\theta)$ is a convex bowl with a single global minimum, which gradient descent reaches from any starting point. In deep networks the non-linear activations of the hidden layers make $J(\theta)$* ***non-convex****: the landscape has many local minima and saddle points, and the solution we reach depends on the initialization of θ. Since non-convex objectives have no closed-form solution, we rely on iterative numerical optimization.*

1.6.2 Regularization

To reduce the risk of overfitting we can add a **regularization term** to the objective:

$$\hat{\theta} = \arg\min_{\theta} \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}(x_i, y_i, \theta) + \lambda R(\theta) \tag{1.33}$$

where $R(\theta)$ penalizes the complexity of the model and λ is a hyperparameter that balances fitting the data against keeping the model simple. Regularization can also be applied implicitly, through architectural or data choices, such as randomly disabling units or connections (dropout) or injecting noise into the inputs.

Note 14. *Networks with smaller weights tend to generalize better than networks with larger weights, because small perturbations or noise in the input produce much smaller fluctuations in the output.*

1.6.3 Minibatch Stochastic Gradient Descent

Computing the exact gradient on the whole dataset (**full-batch** gradient descent) costs $\mathcal{O}(n)$ per step, which is prohibitively slow when n ranges from thousands to billions of examples. **Stochastic Gradient Descent** (SGD) removes the bottleneck by estimating the gradient on a small random subset of the data at each step.

At one extreme, **online learning** ($m = 1$) updates the weights after a single example: updates are immediate, but the estimate is very noisy and does not exploit the parallelism of GPUs. In practice we use **mini-batch SGD**: we draw a random subset of m examples (typically $32 \leq m \leq 512$) and compute their average gradient,

$$g = \frac{1}{m} \sum_{i=1}^{m} \nabla_\theta \mathcal{L}(x_i, y_i, \theta) \tag{1.34}$$

followed by the update

$$\theta = \theta - \eta g \tag{1.35}$$

A complete pass over the n training examples (n/m mini-batch steps) is called an **epoch**.

Why do small mini-batches work so well? The mini-batch gradient g is an **unbiased** estimator of the true gradient, $\mathbb{E}[g] = \nabla_\theta J(\theta)$, and its standard deviation (the gradient noise) shrinks as $1/\sqrt{m}$. This square-root law means diminishing returns for large batches: using $100\times$ more examples per step costs $100\times$ more computation but reduces the noise only by $10\times$. Taking many fast, slightly noisy steps with a moderate batch converges much faster in wall-clock time than taking one slow, exact step. The choice of m is therefore a trade-off: larger batches give more accurate gradients but slower convergence, smaller batches give noisier gradients but faster convergence.

1.7 Backpropagation

Gradient descent needs the gradient of the loss with respect to every weight and bias of the network. **Backpropagation** computes it efficiently by applying the **chain rule of calculus** recursively, from the output layer back to the input layer, reusing at each step what was computed for the layer above.

1.7.1 Chain rule

If $\mathcal{L}$ depends on z, which in turn depends on w, then

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial w} \tag{1.36}$$

1.7.2 The backward pass step by step

We follow the computation on a network with one hidden layer and a sigmoid output unit, where $z^{(1)} = W^{(1)T}x + b^{(1)}$, $h = g(z^{(1)})$, $z = w^{(2)T}h + b^{(2)}$ and $\hat{y} = \sigma(z)$.

1. Output layer. First we compute how the loss changes with respect to the logit z:

$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \tag{1.37}$$

With a sigmoid output $\hat{y} = \sigma(z)$ and the binary cross-entropy $\mathcal{L} = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$, using $\sigma'(z) = \sigma(z)(1 - \sigma(z)) = \hat{y}(1 - \hat{y})$, everything simplifies to

$$\frac{\partial \mathcal{L}}{\partial z} = \hat{y} - y \tag{1.38}$$

2. Output weights and bias. Since $z = \sum_j w_j^{(2)} h_j + b^{(2)}$, we have $\frac{\partial z}{\partial w_j^{(2)}} = h_j$ and $\frac{\partial z}{\partial b^{(2)}} = 1$, so the chain rule gives

$$\frac{\partial \mathcal{L}}{\partial w^{(2)}} = \frac{\partial \mathcal{L}}{\partial z} \cdot h \quad \text{and} \quad \frac{\partial \mathcal{L}}{\partial b^{(2)}} = \frac{\partial \mathcal{L}}{\partial z} \tag{1.39}$$

3. Back to the hidden layer. To update $W^{(1)}$ and $b^{(1)}$ the error signal travels backward, first through the output weights and then through the derivative of the hidden activation $g'\left(z^{(1)}\right)$:

$$\frac{\partial \mathcal{L}}{\partial h} = \frac{\partial \mathcal{L}}{\partial z} \cdot w^{(2)} \tag{1.40}$$

$$\delta^{(1)} = \frac{\partial \mathcal{L}}{\partial z^{(1)}} = \frac{\partial \mathcal{L}}{\partial h} \odot g'\left(z^{(1)}\right) \tag{1.41}$$

where $\odot$ is the element-wise (Hadamard) product. For a ReLU hidden layer ($h = \max\{0, z^{(1)}\}$), $g'(z_j^{(1)}) = 1$ if $z_j^{(1)} > 0$ and 0 if $z_j^{(1)} \leq 0$.

4. First layer parameters. Finally,

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} = x \cdot \left(\delta^{(1)}\right)^T \quad \text{and} \quad \frac{\partial \mathcal{L}}{\partial b^{(1)}} = \delta^{(1)} \tag{1.42}$$

Note 15. ***Inactive ReLU units receive no update.*** *The gradient of an output weight is proportional to the activation of its hidden unit, $\frac{\partial \mathcal{L}}{\partial w_j^{(2)}} = \frac{\partial \mathcal{L}}{\partial z} \cdot h_j$. A neuron that is inactive in the forward pass ($z_j^{(1)} \leq 0 \implies h_j = 0$) therefore produces a zero gradient, and its outgoing weight $w_j^{(2)}$ is not updated. The same holds for its incoming weights, since $g'(z_j^{(1)}) = 0$.*

1.8 Training in Practice

1.8.1 Weight initialization

Activations decide how the signal flows, but the initialization of weights and biases is just as important: it keeps the signal from exploding or vanishing across the layers and makes training stable. It follows three ideas.

Avoid saturation A unit saturates when its pre-activation $|z|$ is too large (Section 1.3), and then it receives almost no gradient. To avoid saturation at the start of training, inputs should be standardized and the initial weights kept small.

Break the symmetry Weights must **never** be initialized to the same value (such as all zeros). If all the weights of a layer are equal, every unit computes the same output, receives the same gradient and undergoes the same update: the units remain identical clones forever, and a layer of 512 units behaves like a single neuron. Drawing the weights at random breaks the symmetry. **Biases**, instead, can safely start at 0 (which also lets us discover dead neurons), or at a small positive value such as 0.1 for ReLU units, so that they do not start in the inactive region $z < 0$.

Preserve the scale The scale of the random weights must preserve the variance of the activations going forward and of the gradients going backward across all the L layers: if the weights are too large, activations explode exponentially with depth; if they are too small, they shrink to zero. Two standard schemes, which sample from a normal distribution with zero mean and a variance that depends on the number of neurons $n^{[l]}$ of the layers, correspond to the two main families of activations:

- **Glorot (Xavier) initialization** is designed for symmetric, zero-centered activations such as tanh (and the sigmoid), which are roughly linear with slope 1 around the origin. It is a compromise between preserving the forward scale ($1/n^{[l-1]}$) and the backward scale ($1/n^{[l]}$):

$$W^{[l]} \sim \mathcal{N}\left(0, \frac{2}{n^{[l-1]} + n^{[l]}}\right)$$

- **He (Kaiming) initialization** is designed for **ReLU**. Since ReLU zeroes all the negative inputs, it silences about half of the units of each layer and halves the variance of the signal; the factor 2 in the numerator compensates for the lost half:

$$W^{[l]} \sim \mathcal{N}\left(0, \frac{2}{n^{[l-1]}}\right)$$

A good initialization keeps activations and gradients well scaled from the very first step, which allows fast and reliable convergence.

1.8.2 The whole training procedure

We can now put all the pieces together. The weights are randomly initialized. Then, for each training example (x, y) (in practice, for each mini-batch), the network computes a forward pass to obtain $\hat{y} = f(x; \theta)$; the error $\mathcal{L}(\hat{y}, y)$ is evaluated; backpropagation computes the gradient; and all the weights are adjusted in the direction that reduces the error. Repeating this over many epochs gradually minimizes the cost function.

1.9 Regularization

Regularization is a crucial technique in deep learning to prevent *overfitting*, which occurs when a model performs well on training data but poorly on unseen data. Regularization strategies aim to reduce the model's variance at the cost of slightly increasing its bias. This trade-off results in a model that generalizes better to new data. Below we will list some of the regularization techniques that we will go to see:

- Parameters norm penalties;
- Dataset augmentation;
- Noise robustness;
- Early stopping;
- Parameter tying and parameter sharing;
- Multitask learning;
- Bagging and other ensemble methods;
- Dropout;
- Adversarial training;

We need to have a model that is capable to generalize on unseen data, so we have to find the right trade-off in order to avoid overfitting and underfitting.

In the context of deep learning, most regularization strategies are based on **regularizing estimators**. This is done through reducing **variance** at the expense of increasing the **bias** of the estimator.

Note 16. *A good estimator is one that decreases the variance without increasing too much the bias.*

When we are working with deep learning to solve complex tasks, we can encounter different problems. This can increase the complexity of the model and the risk of overfitting. In this case, we can find the best large model and in the end apply a regularization process to reduce the complexity of the model and improve the generalization.

The concept of regularization in machine learning and deep learning is to create models that not only minimize the loss function but also manage the variability and generalization capability of the model. To accomplish this, we can employ the following strategies:

- **Directly**: incorporates prior knowledge or expresses a generic preference for simpler models by directly altering the model or its training process. This is implemented by changing constraints, adding restrictions on the parameter values or changing objective function. The constraints serve to penalize certain values of the parameters;
- **Undirectly**: this method does not manipulate the model directly but alters the training process or data to enhance generalization. This can be done using technique such as data augmentation, introducing noise in the training data or using ensemble methods.

1.9.1 Regularization strategies

Parameters Norm Penalties

The most traditional form of regularization consists in adding a penalty term to the loss function, encouraging smaller or sparser parameter values. It can be formulated as:

$$\hat{J}(\theta) = J(\theta) + \alpha\Omega(\theta) \tag{1.43}$$

where $\alpha \in [0, \infty)$ is a hyperparameter that regulates the contribution of the norm penalties, and $\Omega(\theta)$ is the penalty term. This parameter can change for each layer.

Since we have added a penalty term, the optimizer will try to minimize the loss function and at the same time it decreases some measure of the size of the parameters θ. It affects only a subset of parameters.

Ω usually penalizes only the weights (W) of the affine transformation at each layer, not the bias (b).

This method can be implemented using different norms, here we can see some of them:

- **Sum of the weights** or L_1 **Norm** (L_1): Promotes sparsity pushing all values to 0 penalizing small weights more than big one.
$$\Omega(w) = \sum_{w_j} |w_j| = \|w\|_1 \tag{1.44}$$
- **Sum of the squared weights** or L_2 **Norm** (L_2): penalizes larger weights, encouraging smoothness:
$$\Omega(w) = \sum_{w_j} |w_j|^2 = \|w\|_2 \tag{1.45}$$
- p**-norm** (L_p): for $p < 2$ encourage sparser vector, while larger values of p discourage large weights more:
$$\Omega(w) = \sqrt[p]{\sum_{w_j} |w_j|^p} = \|w\|_p \tag{1.46}$$

  In general, all the $p-$norms are used to penalize large weights more than small ones.

The effect of the L_2 norm is to shrinks weights on features that add noise to the model (feature with high variance but low covariance with target), while L_1 norm provides a solution that is sparse. This property can be seen as a features selection mechanism.

Note 17. *Using L_2 negative and positive weights moves towards 0 depending on the magnitude of the weights, while using L_1 negative and positive weights moves towards 0 regardless of magnitude.*

Another improve that we can add to the previous model are constraints related to regularization term, for example, if we consider the following loss function:

$$\hat{J}(\theta) = J(\theta) + \alpha\Omega(\theta) \tag{1.47}$$

we can add a constraint to the regularization term as follows:

$$\Omega(\theta) \leq k \tag{1.48}$$

where k is a small value. In this way the optimization problem changed because we have to consider also the upper-bound of regularization term.

To solve this problem, we can construct a generalize **Lagrange function**, consisting of the original objective function plus a set of penalties. Each penalty is a product between a coefficient, called a Lagrange multiplier (α), and a function representing whether the constraint is satisfied.

$$\mathcal{L}(\theta, \alpha; X, y) = J(w; X, y) + \alpha(\Omega(\theta) - k) \tag{1.49}$$

Formally the optimization problem will be:

$$\arg\min_{\theta} \max_{\alpha \geq 0} \mathcal{L}(\theta, \alpha) \tag{1.50}$$

To solve this problem we can use techniques that minimize J and then project the solution obtained to the feasible region $\Omega(\theta) - k$. This method prevent to get stuck in local minimum and impose stability on the optimization procedure.

Dataset augmentation

We can get better generalization training model on more data. The main problem is that data is limited also labelling is an extremely tedious task. **Dataset augmentation** provides a cheap easy way to increase the amount of your training data.

Moreover, when you are comparing different algorithms you have to compare on augmented and non augmented dataset to avoid bias.

To implement this technique, there are many algorithms available, but when we use them, it's important to be cautious about introducing bias or altering the output in the dataset.

Noise robustness

Noise injection can be thought of as a form of regularization. When we add a noise with infinitesimal variance at input of the model is equivalent to imposing a penalty on the norm of the weights.

Noise can be injected at different levels of deep models, if we add on input we are defining a data augmentation procedure, so we have to use some of the best practices explained before.

Adding small noise pushes model into regions where the model is insensitive to small variations in weights, finding points that are not merely minima, but minima surrounded by flat regions.

Since some dataset present mistake on the labels, we can remedy this problem by adding noise to the labels. This can be done through setting a probability ε for which we think the labels are correct.

This method implemented analytically in the **cross entropy** function. Another example of this technique is **label smoothing**, where we use softmax activation function in the output layer to get a probability.

$$\begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}_{label} \Longrightarrow \begin{bmatrix} 0.9 \\ 0.01 \\ \vdots \\ 0.02 \end{bmatrix}_{label}$$

Using a softmax we will never predict 0 or 1 but it will predict a probability so model can always learn and make more extreme prediction forever. This has the advantage of preventing the pursuit of hard probabilities without discouraging correct classification.

Multitasking learning

Multitasking learning is a way to improve generalization by pooling the examples arising out of several tasks. It is usually performed through the following architecture:

- *Task-specific parameters*: which only benefit from the examples of their task to achieve good generalization.
- *Generic parameters*: parameters shared across all tasks which benefit from the pooled data of all tasks.

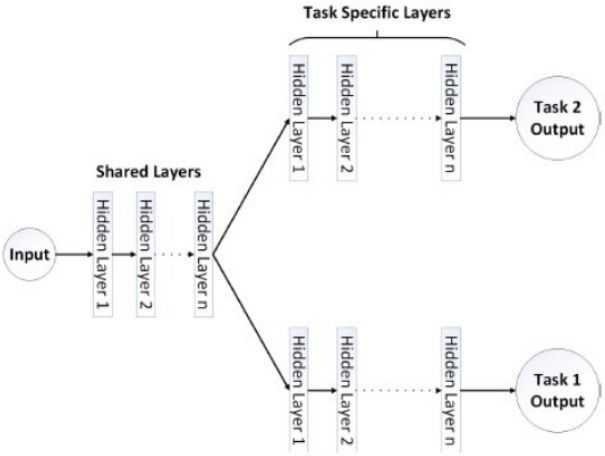


Figure 1.5: Multitasking learning

Note 18. *Multitasking learning is a form of parameter sharing.*

Shared parameters improve generalization in proportion of number of examples for the general task. Additional task imposes constraints on the parameters in the shared layers preventing overfitting.

Note 19. *Improvement in generalization only occurs when there is something shared across the tasks at hand.*

Early stopping

When training large models with sufficient representational capacity to overfit the task, we often observe that training error decreases over time but validation error begins to rise again. We can obtain a model with better validation error by returning to the parameter setting at the point in time with the lowest validation error.

Early stopping is one of the most common techniques used. We can think at this technique as a hyperparameter selection method, where training time is the hyperparameter to be chosen.

Early stopping can be implemented as follow:

1. Choose number of iterations n
2. Train model for n iterations
3. Validate model and compute validation error, compare consecutive validation error and return to point 2 until delta is insignificant or negative.

Every iteration model improves its weights until it starts to going in overfitting.

Note 20. *It's impossible to write all parameters on disk during training.*

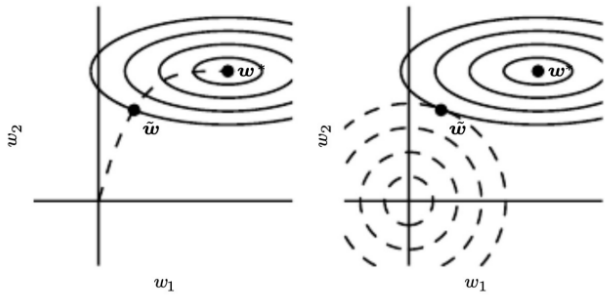


Figure 1.6: On the left we can see the process of early stopping, while on the right we can see a classic regularization process.

Parameters Tying

Parameters Tying refers to explicitly forcing the parameters of 2 models to be close to each other using the norm penalty:

$$\|w(A) - w(b)\| \tag{1.51}$$

with A and B respectively first and second model. This norm penalty is added to the objective function like a constraint. It is used in Siamese networks, convolution operators and multitasking learning.

Bagging

Bagging is a technique for reducing generalization error combining several models (**ensemble model**).

You have to train k different models on k different subsets of training data with same number of examples as original dataset using a random sampling with replacement.

Every model is then tested, they produce k votes for the final result. These models are called ensemble.

The objective of this method is to introduce generalization by using different models trained on different examples using different initialization hyperparameter and different structures to have independent errors.

Cons of this method is that ensemble models doesn't provide us a scalable way to improve performance, moreover we have 2-3 models inside the ensemble.

Dropout

Dropout provides a computationally inexpensive but powerful method of regularizing a broad family of models. It provides an inexpensive approximation of a ensemble with a high number of sub-networks of the starting one.

All the sub-networks should be formed by removing non-output units from underlying base network, generating an exponential number of models, using a treatable amount of memory.

This technique eliminates the need to accumulate model votes in the inference phase. It is intended to train the model to be learned by switching on and off with different probabilities input neurons or neurons in the hidden layers.

To do a train phase we use a **minibatch-based learning algorithm** that makes small steps, such as stochastic gradient descent.

Each time we change minibatch we **randomly sample** a different binary mask to apply to all input and hidden units in the network. Typically the distribution use for sampling is not uniform. For example, the probability of keeping a hidden unit is 0.5, while the probability of including an input unit is 0.8.

We can have some problem, indeed, at training time we are required to divide the output of each unit by the probability of that unit's dropout mask, so we assign as likelihood on the output. In this way we make sure that the expected total input to a unit at test time is the same as the expected total input of that unit at train time.

We haven't theoretically basis for the accuracy but empirically it perform well. Complexity of dropout is limited to a $\mathcal{O}(n)$ computation per example to update because it needs to generate a total of n random numbers and multiply them by the state.

Dropout can be used with every model and also with stochastic gradient descent. The cost of applying dropout to the entire model could be significant, but the amount of benefit on generalization error couldn't be enough to the cost of the regularization.

Adversarial training

We can add **adversarial examples** (inputs perturbed by noise) to train the model for robustness. These are used to train the network and will allowed us to reduce test errors.

Adversarial examples can be generated by adding to a bit of perturbation on some train example with a low multiplication factor of noise. This method allowed us to generate new examples for training from the original train set with slightly changes that are unrecognized by humans.

1.10 Generative Adversarial Models

Generative Adversarial Network (GAN) are generative model that implicitly learn the **data distribution** from a zero-sum game between 2 computing neural network:

- **Discriminator**: a neural network that classify examples between fake and real.
- **Generator**: generate fake example that are given as input to the discriminator.

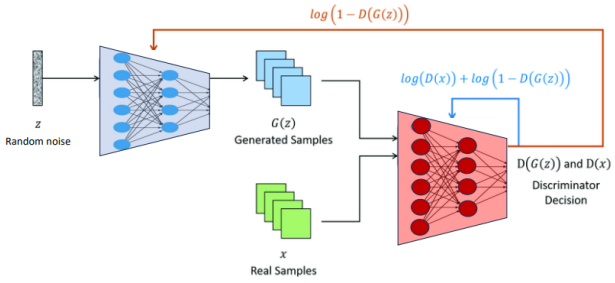


Figure 1.7: Generative Adversarial Network

The target of the generator is to generate example the most similar to real one in such a way to be classified in real example by the discriminator. Quite the opposite the discriminator have to identify real examples from fake one. If the discriminator makes a mistake so we will update discriminator's weights, vice versa we will update generator's weights.

1.11 Optimization for training deep models

When we are creating a deep learning model, we have to consider different aspects to optimize the training process. In this section, we are going to see some issues that can arise during the training process and how to solve them.

Let's start by introducing the difference between **optimization** and **machine learning**. In the first case, we are trying to minimize a function acting directly on it, while in the second case we are trying to minimize a loss function that is a proxy for the true function we want to minimize.

In machine learning, we aim to optimize a performance measure P, based on the test set. This is done by optimizing a different function $J(\theta)$, with the hope that improvements in $J(\theta)$ lead to improvements in P.

Typically, the loss function is a simple average over the training set and can be written as:

$$J(\theta) = E_{(x,y)\approx\hat{P}_{data}}\mathcal{L}(f(x,\theta),y) \tag{1.52}$$

where $\mathcal{L}$ is the loss function, f is the model, x is the input, y is the target and θ are the parameters of the model. Also, $\hat{P}_{data}$ is the empirical distribution of the data.

If we know the true data distribution P_{data}, the loss function, also called the **risk function**, can be expressed as:

$$J(\theta) = E_{(x,y)\approx P_{data}}\mathcal{L}(f(x,\theta),y) \tag{1.53}$$

and this reduce the machine learning problem to an optimization problem.

The problem of using the empirical distribution is that the model is prone to overfitting, because we don't have enough data to represent the real distribution.

Instead of minimizing the empirical risk, we can minimize a **surrogate loss function**. This function is a proxy for the true loss function and it is easier to optimize.

Another difference is that training algorithms doesn't halt at local minima, but when the early stopping criterion is met. This can be roughly thought of as a way to reincorporate the true loss function in the learning process.

1.11.1 Batch and Minibatch

One aspect of machine learning algorithms that separates them from general optimization algorithms is that the objective function usually decomposes as a sum over the training examples. Computing this expectation exactly is very expensive, since it requires evaluating the model on every example in the entire dataset.

However, we can compute these expectations by randomly sampling a small number of examples from the dataset at every iteration. This works because we are computing an expected value and, the **standard error** of the mean estimated from a sample of size m is:

$$SE(\mu) = \sqrt{VAR\left[\frac{1}{m}\sum x^{(i)}\right]} = \frac{\sigma}{\sqrt{m}} \tag{1.54}$$

where σ is the standard deviation of the distribution. This equation shows that the relationship is sub-linear between the number of examples and the standard error.

Another consideration that motivates the estimation of the gradient from a small number of samples is the redundancy in the training set.

Optimization algorithms that use the entire training set to compute the gradient are called **batch** or **deterministic gradient methods**. While the ones that use a single training example for that task are called **stochastic gradient methods**. Most of the algorithms we use for deep learning fall somewhere in between and they are called **mini-batch** or **mini-batch stochastic methods**. We can see the difference between these methods in the following figure 1.8.

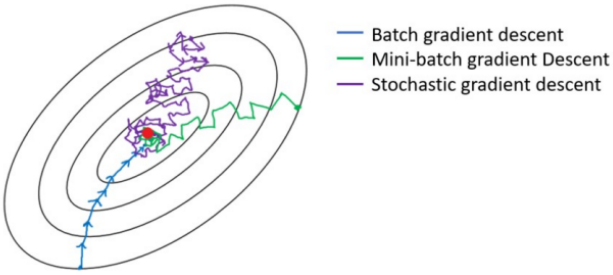


Figure 1.8: Difference between batch, mini-batch and stochastic gradient methods.

In order to chose the right size of the mini-batch, we have to consider that a larger size will provide a more accurate estimate of the gradient, but with less than linear improvement in the standard error. While a smaller

size will provide a less accurate estimate of the gradient, but with more than linear improvement in the standard error and doesn't use the full computational power of the GPU.

Note 21. *Usually, a good choice is a power of 2.*

It is extremely crucial that mini-batches are sampled at **random**, because computing an unbiased estimate of the expected gradient from a set of samples requires that those samples be independent.

When the order of elements in the dataset holds some significance, it is absolute necessity to shuffle the examples before selecting every mini-batch.

1.11.2 Challenges in deep learning

Traditionally, machine learning has avoided this difficulty by carefully designing the objective function and constraints to insure the problem is convex. However, for training deep models we usually face non-convex optimization.

Ill-conditioning

The first challenge is **ill-conditioning**. This is a problem that arises when the optimization problem is poorly conditioned. This means that the curvature of the loss function is very different in different directions.

This condition manifests in stochastic gradient descent when small steps increase the loss function.

This problem can be identify by looking at the **condition number** of the Hessian matrix. If the condition number is very large, we have an ill-conditioned problem.

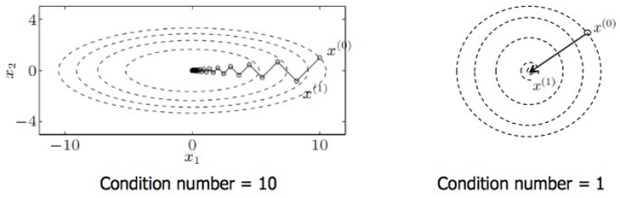


Figure 1.9: Ill-conditioning

Local minima

Functions involved in deep models are guaranteed to have an extremely large number of local minima. However, we will see that local minima are not necessarily a major problem because plateaus are an acceptable solution.

Neural networks aren't **identifiable** there is no single set of parameters from a single training set. In fact, if we have m layers with n neurons each, there are $n!^m$ ways to arranging the hidden units to obtain equivalent solutions. This is called **weight space symmetry**.

Model identifiability issues mean that there can be an extremely large or even uncountably infinite amount of local minima in the cost function of deep models. However, these local minima are all equivalent in value and are not a problematic form of non-convexity.

Local minima are only truly problematic if they have a much higher cost than the global minimum.

Saddle points

For many high-dimensional non-convex functions, local minima and maxima are in fact rare compared to **saddle points**.

A saddle point is a point where the Hessian matrix has both positive and negative eigenvalues and the gradient is zero. We can think of a saddle point as being a local minimum along one cross-section of the cost function and a local maximum along another.

This points are more common in high-dimensional spaces than local minima and they are a problem for optimization algorithms because they are points where the gradient is zero but they are not a minimum.

For first-order optimization algorithms that use only gradient information, the situation is unclear. The gradient can often become very small near a saddle point. On the other hand, gradient descent empirically seems to be able to escape saddle points in many cases. While, for Newton's Method, saddle points constitute a major problem, because it actively seeks solutions at critical points where the gradient is zero.

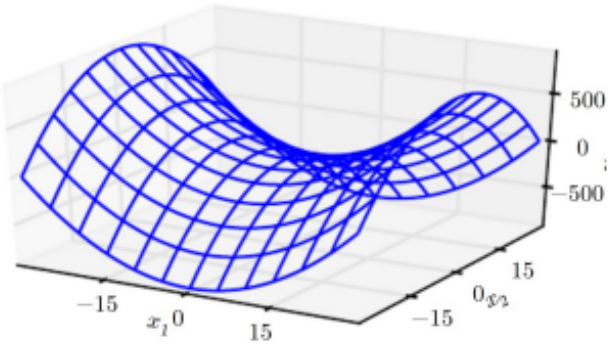


Figure 1.10: Saddle point

Plateaus

Another problem that can arise during the training process is the presence of **plateaus** or **flat regions**. These are regions where the gradient is very small and the optimization algorithm can't make progress.

Cliffs and exploding gradients

Neural networks with many layers often have extremely steep regions reassembling cliffs. This regions are the result from the multiplication of many large weights together.

An extremely steep cliff structure, the gradient update step can move the parameters extremely far, usually jumping off of the cliff structure altogether.

A solution to this is to use the **gradient clipping heuristic** which intervenes to reduce the step size to be small enough that it is less likely to go outside the region where the gradient indicates the direction of approximately steepest descent.

Additional Problems

There are other problems that can arise during the training process, such as:

- **Long Term Dependencies**: arises when the computational graph is very deep. The result of this problem is vanishing and exploding gradients.
- **Inexact Gradients**: we usually only have a noisy or even biased estimate of the gradient and the Hessian. Sometimes, gradients for our loss functions are even intractable. Not a big issue in neural network training. Surrogate loss functions tend to perform well enough in practice.
- **Poor Correspondence between Local and Global Structure**: it is possible to overcome all of the above problems at a single point and still perform poorly if the direction that results in the most improvement locally does not point toward distant regions of much lower cost.

Note 22. *The initialization is very important.*

1.11.3 Optimization algorithms

There are many optimization algorithms that can be used to train deep models. In this section, we are going to see some of them.

Stochastic Gradient Descent

Stochastic Gradient Descent (SGD) is the most used optimization algorithm in deep learning. It is based on the idea of computing the gradient of the loss function on a mini-batch and then update the weights of the model. We can describe the algorithm as follows:

- Sample a mini-batch of examples from the training set.
- Compute the gradient of the loss function on the minibatch.

$$g \leftarrow \frac{1}{m} \nabla_\theta \sum_{i=1}^{m} \mathcal{L}(f(x^{(i)}; \theta), y^{(i)}) \tag{1.55}$$

- Update the weights of the model using the gradient.

$$\theta \leftarrow \theta - \epsilon_k g \tag{1.56}$$

It is important to note that the learning rate ϵ_k is a hyperparameter that must be adaptive. This is because the learning rate is a very important hyperparameter and it can affect the convergence of the algorithm. A sufficient condition for convergence is:

$$\sum_{k=1}^{\infty} \epsilon_k = \infty \quad \text{and} \quad \sum_{k=1}^{\infty} \epsilon_k^2 < \infty$$

A common way to implement an adaptive learning rate is to use a linear decay until the iteration τ. This can be done as follows:

$$\epsilon_k = (1 - \alpha)\, \epsilon_0 + \alpha \epsilon_\tau \tag{1.57}$$

In this case we have to choose three hyperparameter: ϵ_0, ϵ_τ and α. Usually, ϵ_τ is chosen to be 1% of ϵ_0.

To choose ϵ_0 we can try to make different run for a fixed number of iterations and choose the best one.

Momentum

Momentum is a method that helps accelerate SGD in the relevant direction and dampens oscillations. It is a method that accumulates an exponentially decaying average of past gradients and continues to move in their direction.

The update rule for momentum is:

$$\theta \leftarrow \theta + v \tag{1.58}$$

where v is the velocity vector and it is defined as:

$$v \leftarrow \alpha v - \epsilon \frac{1}{m} \nabla_\theta \sum_{i=1}^{m} \mathcal{L}(f(x^{(i)}; \theta), y^{(i)}) \tag{1.59}$$

where α is the momentum parameter and it is usually set to 0.9.

This algorithm aims to solve the problem of poor conditioning of the Hessian matrix, and the variance of stochastic gradients.

The momentum algorithm can be seen as a ball rolling down a hill. The ball gathers momentum as it rolls down the hill, and it continues to move in the direction of the momentum even if it encounters a small hill.

Delta Bar Delta

Delta Bar Delta is a method that adapts the learning rate for each parameter. It is based on the following idea:

- if the partial derivative of the loss function with respect to a parameter remains the same, increase the learning rate.
- otherwise, if that partial derivative changes sign, decrease the learning rate.

AdaGrad

AdaGrad is a method that adapts the learning rate for each parameter. This method scale the gradient according to the historical norms. In this way, learning of parameters with high partial derivatives decrease faster.

This method is based on the following idea:

- Accumulate the square of the gradient into a vector r.

$$r \leftarrow r + g \odot g \tag{1.60}$$

- Update the weights element wise.

$$\Delta\theta \leftarrow -\frac{\epsilon}{\sqrt{r + \epsilon}} \odot g \tag{1.61}$$

- Update parameters

$$\theta \leftarrow \theta + \Delta\theta \tag{1.62}$$

RMSProp

RMSProp is a method that adapts the learning rate for each parameter. This method is a modification of AdaGrad to perform better on non-convex problems. The main difference is that RMSProp use an exponentially weighted moving average. This method is based on the following idea:

- Accumulate the square of the gradient into a vector r.

$$r \leftarrow \rho r + (1-\rho) g \odot g \tag{1.63}$$

- Update the weights element wise.

$$\Delta\theta \leftarrow -\frac{\epsilon}{\delta + \sqrt{r}} \odot g \tag{1.64}$$

- Update parameters

$$\theta \leftarrow \theta + \Delta\theta \tag{1.65}$$

Adam

Adam is a variation of RMSProp in which we add the momentum. Also, it add a bias correction to the moments to account for their initialization at the origin. This method is based on the following idea:

- Update time step:

$$t \leftarrow t + 1 \tag{1.66}$$

- Update biased moment estimates

$$\begin{aligned} s &\leftarrow \rho_1 S + (1-\rho_1) g \\ r &\leftarrow \rho_2 r + (1-\rho_2) g \odot g \end{aligned} \tag{1.67}$$

- Correct biases

$$\begin{aligned} \hat{s} &\leftarrow \frac{S}{1-\rho_1^t} \\ \hat{r} &\leftarrow \frac{r}{1-\rho_2^t} \end{aligned} \tag{1.68}$$

- Update parameters

$$\Delta\theta \leftarrow -\epsilon \frac{\hat{s}}{\delta + \sqrt{\hat{r}}} \tag{1.69}$$

- Update model params:

$$\theta \leftarrow \theta + \Delta\theta \tag{1.70}$$

Chapter 2

Autoencoder

The **autoencoder** are neural networks designed to learn efficient representations of data, often for dimensionality reduction or feature extraction. They are trained to attempt to copy its input to its output. This is achieved with a network divided into two parts:

- An **encoder** that maps the input to a hidden representation (latent space);
- A **decoder** that reconstructs the input from the hidden representation.

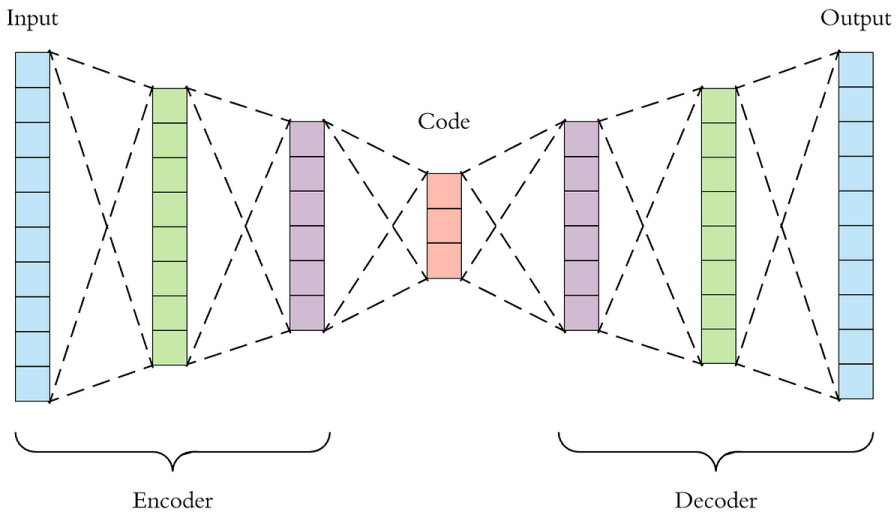


Figure 2.1: Autoencoder architecture

An autoencoder can be mathematically expressed as $h = f(x)$, where h is the latent representation of the input x. The output of the autoencoder is given by $\hat{x} = g(h) = g(f(x))$, where g is the decoder function. The autoencoder is trained to minimize the reconstruction error, i.e., the difference between the input x and its reconstructed output $\hat{x}$, aiming for $\hat{x} \approx x$.

This structure forces the model to select which aspects of the data to preserve, thereby learning potentially useful properties of the data.

2.1 Types of Autoencoder

2.1.1 Undercomplete Autoencoder

An **undercomplete autoencoder** has a latent space (hidden layer) that is smaller in dimensionality than the input. This design forces the autoencoder to compress the input, encouraging it to capture the most significant features of the data.

This model is trained to minimize the reconstruction error, i.e. the difference between the input and the output. The loss function is usually the *mean squared error* (MSE) between the input and the output.

$$\mathcal{L}(x, g(f(x))) = ||x - g(f(x))||^2 \tag{2.1}$$

Note 23. *If the activation functions in the encoder and decoder are linear, an undercomplete autoencoder is equivalent to Principal Component Analysis (PCA).*

We could have non linear autoencoder, where the encoder and decoder are non-linear functions. This allows the autoencoder to learn more complex representations of the data.

2.1.2 Regularized Autoencoder

Regularized autoencoder are autoencoder that are trained to minimize the reconstruction error, but also have a regularization term. This term is used to prevent the autoencoder from learning the identity function.

Regularized autoencoders can be further categorized as follows:

- **Sparse autoencoder**: is a regularized autoencoder where the hidden layer is greater than the input layer. Usually we add a regularization term to the loss function to penalize the activation of the hidden units. This type must respond to statistical features of the dataset, rather than acting as an identity function.
- **Denoising autoencoder**: are basic autoencoder where the task is to reconstruct the input without noise. The input is corrupted by adding some noise, and the autoencoder is trained to reconstruct the original input.
- **Stacked autoencoder**: are autoencoder where the encoder and decoder are composed of multiple layers. This allows the autoencoder to learn more complex representations of the data.
- **Variational Autoencoder**: are autoencoder that are trained to learn the parameters of the probability distribution that generates the data. This allows the autoencoder to generate new data similar to the training data. The middle layer of the autoencoder is used to represent the mean and the variance of the distribution. And, the loss function need to be improved to take into account the difference between the probability distribution of the input and the output.

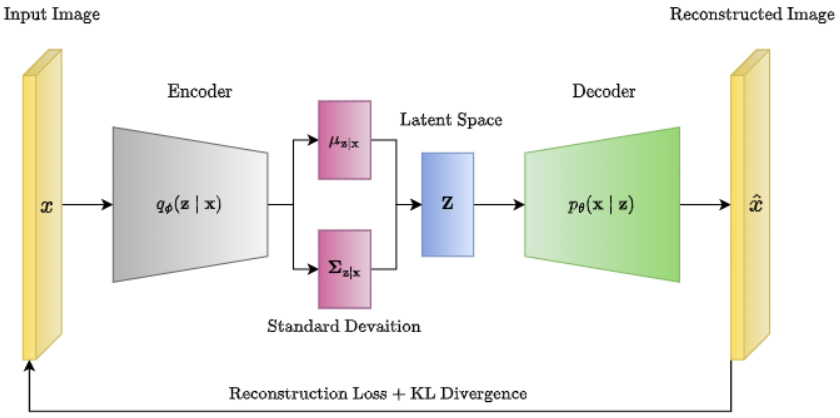


Figure 2.2: Variational Autoencoder architecture

Chapter 3

Convolutional Neural Network

Convolutional Neural Network (CNN) are particular types of neural network feed forward. Their goal is to use multiple layers, stacked one on top of the other, in order to extract input-related features. These layers are organized in a hierarchical way, where the lower level calculates more global feature that can be used in another task, while higher layer learns more specific features related to the task.

Unlike the networks seen so far, CNNs organize weights using 3 dimensions as shown in Figure 3.1. From this point on, we will talk about the weights also considering the **volume**. These volumes arise because the input consists of spatial dimensions (image width and height) along with multiple input channels (e.g., RGB for color images).

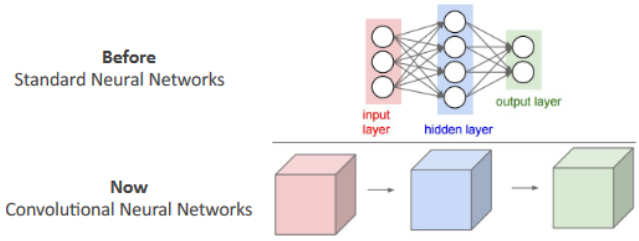


Figure 3.1: Weights representation in Feed Forward vs CNN.

In a CNN each neuron acts as a filter, processing the entire spatial dimensions and all input channels simultaneously. The output of each layer is another volume, where the depth corresponds to the number of filters (or neurons) in the hidden layer.

This particular type of network is widely used in the field of digital signal processing, such as image or audio signals. If we look at image processing, this kind of neural network offers a much more cost-effective approach than traditional feed forward.

The most significant difference lies in how connections are structured. In a traditional feed forward network, each input neuron is fully connected to all neurons in the next layer. This results in a significant complexity challenge. For instance, with an image of 100×100 pixels, the input to the network is a vector of 10000 pixels, leading to an enormous number of connections.

CNNs, on the other hand, introduce local connectivity, where each neuron focuses on a small, localized portion of the image in each channel (**Local connectivity**) as shown in Figure 3.2.

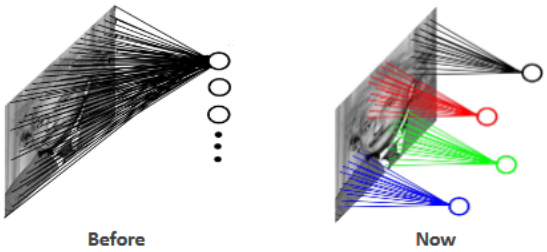


Figure 3.2: Difference between the connections of neurons in a feed forward network and a CNN.

This drastically reduces the number of neurons and connections required. For example, if the input image has dimensions $1000 \times 1000 \times 3$ (width, height, and 3 color channels), a single neuron could use a $5 \times 5 \times 3 = 75$ parameters. This is because the neuron processes a 5×5 spatial region across all channels of the previous layer, a property referred to as **full depth** connectivity.

Also, another difference is that now, we have neurons organized in depth, we have multiple neurons all looking the same region of the input volume, stacked along depth dimension. This is shown in Figure 3.3.

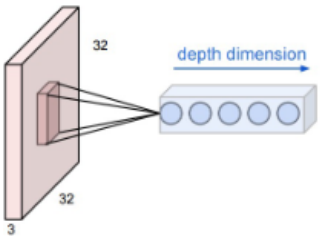


Figure 3.3: Organization of neurons in CNN

In total we have $5 \times 5 \times 3$ weights to learn shared between each patch of the image, this reduce a lot the number of learning parameter compared to feed forward network. This concept is called **weights sharing**, which means that the weights in the layer are shared across spatial positions.

In general, the structure of a CNN is organized so that it has a first part, which includes convolutional layers, that extract the main characteristics from the input. There is then a second part, composed of fully connected layer, which performs the actual classification task. We can then represent this type of network with a structure like the one shown in Figure 3.4.

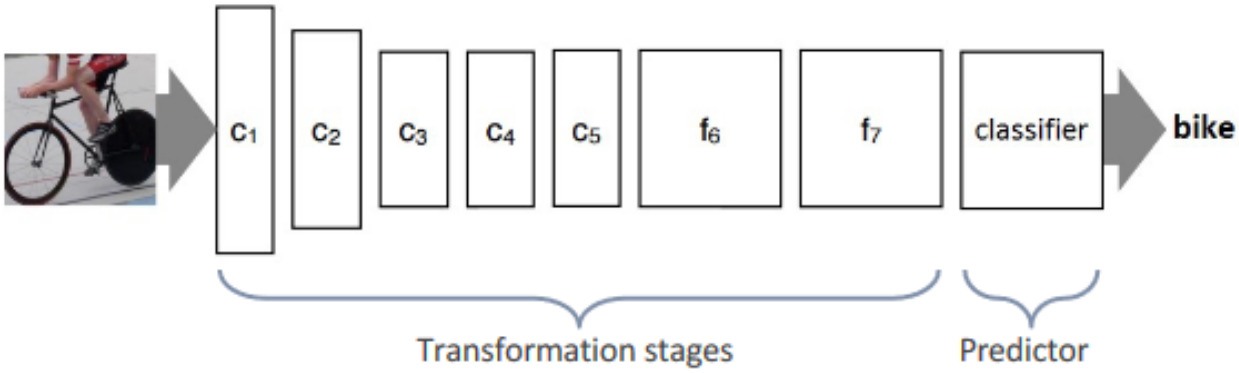


Figure 3.4: Overall CNN architecture.

The concept of **backpropagation** can still be utilized with this architecture, which is a crucial aspect of training CNNs.

Respect to traditional neural networks, CNNs incorporate additional specialized layers, such as **Spatial pooling** and **Local response normalization**.

These layers are useful to reduce computational complexity, increase invariance and ease the optimization.

3.1 CNNs components

3.1.1 Linear Convolution

We begin by introducing the concept behind convolutional networks, namely the concept of **linear convolution**. Convolution is a linear, local (it applies on patches) and translation-invariant operator. To enrich the data representation, we can use a **filter bank**, that is a collection of Q sets of K filters that allows to produce an output of Q channel.

Let's see now, on a more mathematical level, what convolution represents. First we introduce what we need:

- $x = H \times W \times K$ is our input where H represents the height dimension, W the width dimension and K the number of channels.

- $F = H' \times W' \times K \times Q$ is our filter bank where Q is the number of filters that need to be apply to each channel.
- $y = (H - H' + 1) \times (W - W' + 1) \times Q$ is our output.

In addition to these, another very important thing to define is the **stride**. The stride determines the step size for moving the filter across the input. By default, the stride is set to 1, meaning the filter's center moves one pixel at a time before being applied again.

Note 24. *A filter bank with 16 filters of depth K produces an output with 16 channels.*

The convolution is expressed by the following formula:

$$y_{i,j,q} = y_q + \sum_{u=0}^{H-1} \sum_{v=0}^{W-1} \sum_{k=1}^{K} x_{u+i,v+j,k} \cdot F_{u,v,k,q} \tag{3.1}$$

where:

- y_q is the bias of the filter F_q;
- $x_{u+i,v+j,k}$ is the input patch of channel k;
- $F_{u,v,k,q}$ is the filter for the channel k that computes output channel q.

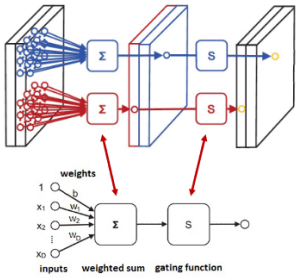


Figure 3.5: Example of convolutional network with a filter bank of two elements.

We can apply filters in 2 way:

- **Lattice structure**: a single filter is applied to a single channel ($K = 1$);
- **Multiple feature structure**: a single filter is applied to all channel of the input image ($K > 1$).

In CNN, after the filter application we use a non-linear activation function, also called **gating function**: *Sigmoid*, *Tanh*, *ReLU* and *SmoothReLU*.

3.1.2 Spatial pooling

We have already said that there are different types of layers in a CNN, one of them is the **spatial pooling**.

The purpose of this layer is to subsample the image, reducing its spatial dimensions and, in turn, the computational load. Additionally, spatial pooling aggregates information from the image, enhancing translation invariance, which makes the model more robust to variations in the exact spatial positions of features. This is done in two main ways:

- **avg pooling**: computing an average of the value of the pixels in the neighborhood:

$$y_{ijk} = \text{avg}_{p,q \in \Omega} x_{p,q,k} \tag{3.2}$$

- **max pooling**: taking the maximum value from the pixels in the neighborhood:

$$y_{ijk} = \max_{p,q \in \Omega_{i,j}} x_{p,q,k} \tag{3.3}$$

This is done channel by channel.

3.1.3 Local Response Normalization

Local Response Normalization layers have the objective of normalizing the effect of contrast to improve the network invariance in this respect. This layer also allows for improved optimization and sparsity (accuracy and speed).

In general, there are two ways of applying it:

- **Within Channel**: operates independently on different feature channels, and also rescales each input feature basing on a local neighborhood.

$$y_{i,j,k} = x_{i,j,k} \left(k + \alpha \sum_{(u,v)\in\mathcal{N}(i,j)} x^2_{u,v,k} \right)^{-\beta} \tag{3.4}$$

- **Across Channels**: operates independently at each spatial location and groups of channels. It also normalizes groups $G(k)$ of feature channels. Groups are usually defined in a sliding window manner.

$$y_{i,j,k} = x_{i,j,k} \left(k + \alpha \sum_{q\in G(k)} x^2_{i,j,q} \right)^{-\beta} \tag{3.5}$$

3.1.4 Input sensibility

CNN are sensible to the input images, moreover we need a lot of data to train from scratch a CNN. So we have to normalize images using:

- **Local mean subtraction**
- **Normalization**

to have data centered in 0. To prevent overfitting we can use:

- Weight decay;
- Dropout;
- Data augmentation: for example changing illuminants, flip the image, random crop and a geometric distortion.

Remember that is always better to accept new data.

3.2 CNN Architectures

Below, we will introduce some of the most famous CNN architectures. This is done because the training from zero of one of these models requires a very large amount of data and many computational resources.

- **LeNet** is one of the first model that this is first create for the MNIST dataset. This network take in input a gray scale image of size 32×32.

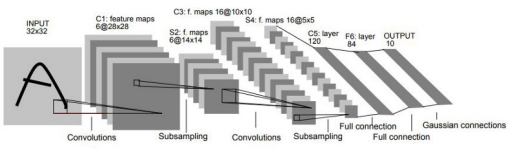


Figure 3.6: LeNet

- **AlexNet** was created for the ImageNet challenge and combines some convolutional layers with feed forward layers. It starts with a large convolutional level that is gradually reduced in order to encode the deeper layers of the more specific information in the task. In general we decrease filter dimension, increasing the depth channels.

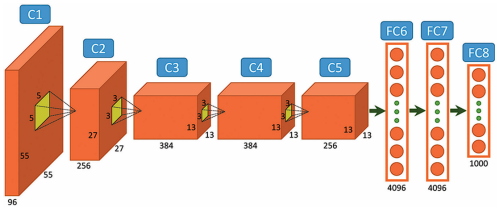


Figure 3.7: AlexNet

- **VGG** is a family of CNNs that consists of networks with varying numbers of convolutional layers. Compared to AlexNet, VGG uses smaller filters to reduce the number of parameters. Additionally, due to the concept of the **receptive field**, using smaller filters in multiple layers can achieve a similar receptive field to using a larger filter in a single layer. This allows VGG to maintain model depth while managing the number of parameters more efficiently.

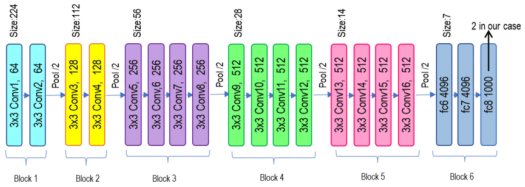


Figure 3.8: VGG-19

- **GoogLeNet** is one of the first networks to introduce the **module** concept. In particular, it uses the module called **inception** which is a part of the network, which can be stacked over other similar modules to get a larger network.

  As we are increasing the size of the network, we may encounter the problem of vanishing gradient. To solve this problem the creators introduce auxiliary output levels in different parts of the network in order to inject the gradient into different levels.

  This is done by creating a loss function which is a linear combination of the output, with an higher weights on final loss than the others losses.

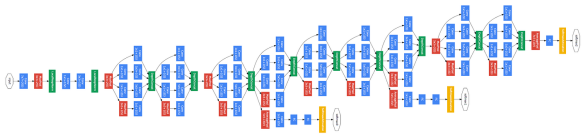


Figure 3.9: GoogLeNet

- **ResNet** is a network type that uses the idea of **residue**. The basic concept is to provide in input the identity function and let the network learn how to modify this function to approach the one you want to learn. The identity function is used because it can be easily implemented by adding the input to the result obtained at a particular point in the network. This is done to limit the vanishing gradient problem as the derivative of the identical function is always 1. With this trick you can implement much deeper networks.

Note 25. *Usually we can use* 1×1 *convolution to squeeze information in depth channel without having impact on the spatial information.*

3.2.1 Transfert learning

As mentioned above, it is not always possible to train a CNN from scratch. Depending on the situation and the amount of data available. Generally we can train model from scratch on a big dataset, then we can fine tune on our task dataset. In general, one of the following techniques may be adopted:

- New dataset is small and similar to original dataset: we can train a linear classifier on CNN features from higher layers;

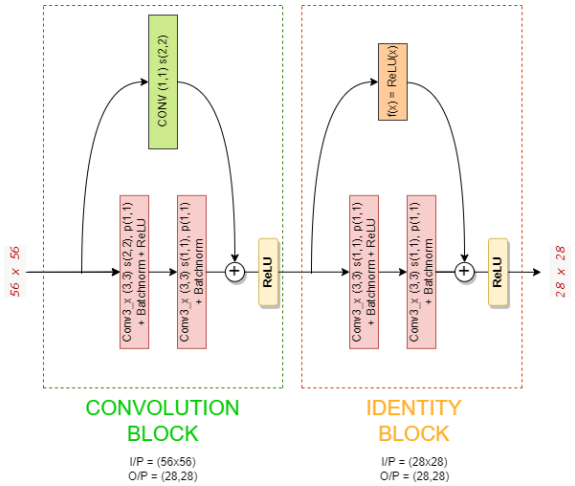


Figure 3.10: ResNet

- New dataset is large and similar to original dataset: we can Fine-tune the CNN;
- New dataset is small but very different from original dataset: we can train a linear classifier on CNN features from lower layers because we have to learn general features;
- New dataset is large and very different from original dataset: we can train CNN from scratch or fine-tune it

Chapter 4

Model Compression

When developing a Deep Neural Network model, it is crucial to consider the devices on which it will run, especially if the model is to be distributed to users. For example, running a large language model (LLM) on a mobile device would not be practical. Therefore, it's important to find the right balance between performance and compatibility. One possible solution is to host the model in the cloud, but this approach introduces challenges, such as network latency and privacy concerns.

An alternative approach is to create a more compact model that can run efficiently on a mobile device while achieving performance similar to that of larger models. An added advantage of this solution is that, due to its reduced size, the inference process will be faster.

To achieve these results, several aspects were studied, between the main approaches we can mention: weight sharing, network pruning, low rank matrix and tensor decomposition, knowledge distillation, quantization, and low resource and efficient architectures.

4.1 Weight Sharing

Weight sharing is a simple form of network reduction that involves sharing weights between layers or structures within layers. Unlike other compression techniques, weight sharing is typically applied before training the original network, rather than compressing the model after training. This technique reduces the network size and avoids sparsity. However, it is not always clear how many and which weights should be shared before performance degradation becomes unacceptable for a given network architecture and task.

An example of weight sharing can be seen in Convolutional layers, where each neuron shares the same set of weights across different spatial locations.

4.2 Network pruning

Pruning weights is one of the most commonly used techniques to reduce the number of parameters in a pre-trained Deep Neural Network. This can lead to reductions in storage and model runtime. Performance is usually maintained by retraining the pruned network.

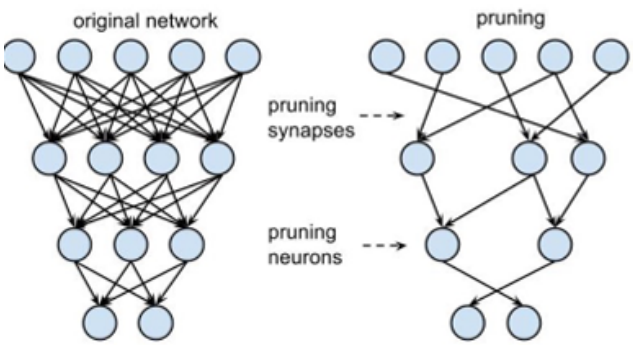


Figure 4.1: Pruning weights in a neural network

Iterative weight pruning prunes weights while retraining the network until the desired trade-off between network size and accuracy is achieved. The simplest pruning strategy involves setting a threshold to determine which weights or units (in this case, the absolute sum of the magnitudes of incoming weights) are removed. The threshold can vary for each layer or be global for the entire network. Instead of setting a threshold, one can predefine a percentage of weights to be pruned based on their magnitude, or a percentage aggregated by weights for each layer. Typically, the weights with magnitudes closest to 0 are removed.

Magnitude-based pruning (MBP) is the most commonly used pruning technique in DNNs due to its simplicity. It works well for a wide range of machine learning models, including DNNs, across various tasks. In general, global MBP tends to outperform layer-wise MBP because it provides more flexibility in determining the amount of sparsity for each layer. This allows more important layers to remain denser while less important layers contain more non-zero entries.

4.2.1 Categorization of Pruning Techniques

- **Magnitude-based (iterative pruning + retraining)**: This technique removes the weights with the lowest absolute values based on a set threshold or percentage, either layer-wise or globally. The steps are as follows:

  1. Choose a neural network architecture.
  2. Train the network until a reasonable solution is obtained.
  3. Prune the weights with magnitudes smaller than a threshold τ.
  4. Retrain the network until a reasonable solution is obtained.
  5. Repeat step 3.

- **Methods that penalize the objective** with a regularization (L_1, L_2, etc.) term to force the model to learn a network with smaller weights and prune the smallest weights.

  This pruning technique is based on weight regularization, where we add a penalty term to the objective function to encourage the model's weights to approach zero. The smallest weights are then deleted.

  Typically, an L_2 regularization is used:

  $$C(\mathbf{w}, \mathbf{v}) = \frac{\varepsilon}{2} \left(\sum_{m=1}^{h} \sum_{l=1}^{n} \mathbf{w}_{ml}^2 + \sum_{m=1}^{h} \sum_{p=1}^{C} \mathbf{v}_{pm}^2 \right)$$

  However, the main issue with using the above quadratic penalty is that all parameters decay exponentially at the same rate and disproportionately penalizes larger weights.

- **Methods that compute the sensitivity of the loss function** when weights are removed, using this sensitivity as a criterion for removing weights that result in the smallest change in loss. **(Diagonal) Hessian-based pruning: Optimal Brain Damage**:

  1. Choose a neural network architecture.
  2. Train the network until a reasonable solution is obtained.
  3. Compute the second derivatives for each parameter.
  4. Compute the saliencies for each parameter: $S_k = \frac{\delta^2 E}{\delta^2 u_k} u_k^2$.
  5. Sort the parameters by saliency and delete low-saliency parameters.
  6. Repeat step 2.

- **Search-based approaches** (e.g particle filters, evolutionary algorithms, reinforcement learning) that seek to learn or adapt a set of weights, links, or paths within the neural network, keeping those that are salient for the task.

4.2.2 Structured vs. Unstructured Pruning

Another important distinction to be made is between structured and unstructured pruning techniques:

- **Structured pruning** aims to preserve network density for computational efficiency (resulting in faster computation at the expense of less flexibility) by removing groups of weights.

- **Unstructured pruning** is unconstrained as to which weights or activations are removed, but the sparsity means that the dimensionality of the layers does not change.

Therefore, sparsity in unstructured pruning techniques generally provides good performance at the expense of slower computation.

Structured pruning via weight regularization

Since standard pruning leads to non-structured connectivity, structured pruning can be used to reduce memory usage and increase speed. Hardware is more amenable to dealing with dense matrix multiplications, especially when there are few non-zero entries in matrices and tensors. Convolutional Neural Networks (CNNs) are particularly suitable for this type of pruning since they consist of sparse connections.

Group sparsity regularization

Group sparse regularizers enforce a subset of weight groupings (such as filters in CNNs) to be close to zero when trained using stochastic gradient descent. For example, consider a convolutional kernel represented as a tensor $K(i;j;s;:)$. The group-wise L_1-norm, L_2-norm is given as:

$$\omega_{2,1}(K) = \lambda \sum_{i,j,s} |\Gamma_{ijs}| = \lambda \sum_{i,j,s} \sqrt{\sum_{t=1}^{T} K(i,j,s,t)^2}$$

Where Γ_{ijs} is the group of kernel tensor entries $K(i;j;s;:)$, where $(i;j)$ represents the pixel location in the i-th row and j-th column of the image for the s-th input feature map. This regularization term forces certain groups Γ_{ijs} to be close to zero, which can be useful for pruning.

4.3 Low rank matrix and tensor decomposition

The idea starts from the observation that most weights are in the fully connected layers, and that fully connected layers are implemented with a single matrix multiplication, i.e. connections from a fully connected layer with N units into one with M units can be stored in a $W = N \times M$ matrix.

Let's now rewrite the matrix W as a product of three matrices U, S and V:

$$W = USV^T$$

where $U \in \mathbb{R}^{m \times m}$ matrix, $S \in \mathbb{R}^{m \times k}$ diagonal matrix and $V \in \mathbb{R}^{k \times k}$ matrix.

By only keeping the $t(< k)$ singular values with largest magnitude:

$$\tilde{W} = \tilde{U}\tilde{S}\tilde{V^T}$$

where $U \in \mathbb{R}^{m \times t}$ matrix, $S \in \mathbb{R}^{t \times t}$ diagonal matrix and $V \in \mathbb{R}^{t \times k}$ matrix. So the number of parameters is reduced from $m \times k$ to $m \times t + t + t \times k$.

4.4 Knowledge distillation

Involves learning a smaller network from a large network using supervision from the larger network and minimizing the entropy, distance or divergence between their probabilistic estimates. Probably the first work about KD is from Bucilua et al. where they explored the idea of reducing model size by learning a student network from an ensemble of models.

- They use a teacher network to label a large amount of unlabeled data and train a student network using supervision from the pseudo labels provided by the teacher;
- They find performance is close to the original ensemble with 1000× smaller network.

Hinton et al. showed a neural network knowledge distillation approach where a small model is trained using supervision (class probability outputs) from the original "teacher" model. They showed that learning from the larger network outperformed the smaller network learning from scratch in the standard supervised classification setup.

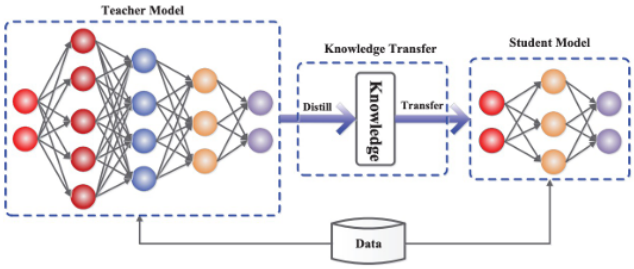


Figure 4.2: Knowledge Distillation

4.5 Quantization

Quantization is the process of representing values with a reduced number of bits. In neural networks, this can be applied to weights, activations and gradient values. Typically, when training on the GPU, values are stored in 32-bit floating point (FP) single precision. Half-precision for floating point (FP-16) and integer arithmetic (INT-16) are also commonly considered. INT-16 provides higher precision but a lower dynamic range compared to FP-16. In FP-16, the result of a multiplication is accumulated into a FP-32 followed by a down-conversion to return to FP-16.

To speed up training, faster inference and reduce bandwidth memory requirements, ongoing research has focused on training and performing inference with lower-precision networks using integer precision (IP) as low as INT-8, INT-4, INT-2 or 1 bit representations.

Designing such networks makes it easier to train such networks on CPUs, FPGAs, ASICs and GPUs. Two important features of quantization are the range of values that can be represented and the bit spacing.

For the range of signed integers with n bits, we represent a range of $[-2n-1, 2n-2]$ and for full precision (FP-32) the range is $\pm 3.4e38$. For signed integers, there are $2n$ values in that range and approximately $4.2e9$ for FP-32. FP can represent a large array of distributions which is useful for neural network computation, however this comes at larger computational costs when compared to integer values.

For integers to be used to represent weight matrices and activations, a FP scale factor is often used hence many quantization approaches involve a hybrid of mostly integer formats with FP-32 scaling numbers. This approach is often referred to as mixed-precision (MP). Different MP strategies have been used to avoid overflows during training and/or inference of low resolution networks given the limited range of integer formats.

4.6 Low resource and efficient architectures

- **MobileNet**: is a sort of compression of CNN for embedded mobile vision application. They use depth-wise separable convolution (DSC) and 2 hyperparameter that trade off latency and accuracy. DSC split the convolution in 2 steps, first filtering then combining outputs of each DSC filter, this is why it is referred as factorization approach.

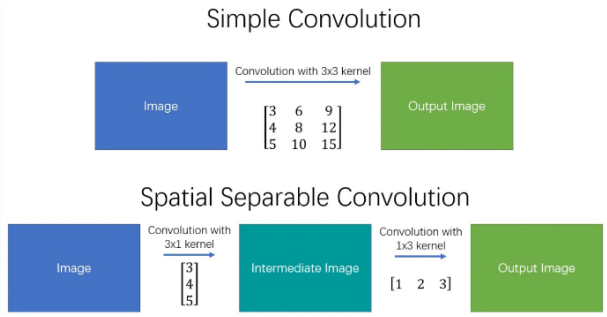


Figure 4.3: Spatial Separable Convolution

- **SqueezeNet**: reduce the network architecture by reducing 3×3 filters to 1×1 filters (squeeze layer). Reduce the number of input channels to 3×3 filters using squeeze layers and down sample later in the network to avoid the bottleneck of information through the network too early and in turn lead to better

performance. A fire module is made up of the squeeze layer and an expand layer that is a mix of 1×1 and 3×3 convolution filters. The number of filters per fire module is increased as it gets closer to the last layer.

- **ShuffleNet**: uses point-wise group convolutions (i.e using a different set of convolution filter groups on the same input features, this allows for model parallelization) and channel shuffles (randomly shuffling helps information flow across feature channels) to reduce compute while maintaining accuracy. ShuffleNet is made up economical 3×3 depth-wise convolutional filters and replace 1×1 layer with point-wise group convolutional followed by the channel shuffle. Unlike predecessor models, ShuffleNet is efficient for smaller networks.

- **DenseNet**: Gradients can vanish in very deep networks because the error becomes more difficult to backpropagate as the number of matrix multiplications increases. DenseNets address gradient vanishing connecting the feature maps of the previous layer to the inputs of the next layer, similar to ResNet skip connections. This reusing of features means the network efficient with its use of parameters. Although, deep and thin DenseNetworks can be parameter efficient, they do tradeoff with memory/speed efficiency in comparison to shallower yet wider network because all layer outputs need to be stored to perform backpropagation. However, DenseNets too can be made wider and shallower to become more memory efficient if required.

Chapter 5

Recurrent Neural Networks

Recurrent Neural Networks (RNN) are a family of neural networks for processing sequential data. It is specialized for processing sequence of values $x^{(1)}, \ldots, x^{(\tau)}$. Recurrent networks can scale to much longer sequences than would be practical for networks without sequence-based specialization. Most recurrent networks can also process sequences of variable length.

To go from multilayer networks to recurrent networks, we need to take advantage of one of the early ideas: sharing parameters across different parts of a model. Parameter sharing makes it possible to extend and apply the model to examples of different forms (different lengths, here) and generalize across them.

Recurrent networks share parameters differently compared to CNNs. Each output member is a function of the previous output members and is produced using the same update rule applied recursively to the earlier outputs. This recurrent formulation leads to parameter sharing across a very deep computational graph.

Usually, RNN are defined using computational graph (CG) (Figure 5.1a). In particular, in the Figure 5.1a we used the following notation:

- $x^{(i)}$ is a value of the input sequence;
- $h^{(i)}$ represent the hidden layers;
- $o^{(i)}$ represent the output layer;
- $y^{(i)}$ represent the training target;
- $L^{(i)}$ is the loss that measures how far each o is from the corresponding training target y.

The RNN has input-to-hidden connections parametrized by a weight matrix $\mathbf{U}$, hidden-to-hidden recurrent connections parametrized by a weight matrix $\mathbf{W}$, and hidden-to-output connections parametrized by a weight matrix $\mathbf{V}$.

RNN uses the information coming from previous hidden layers to compute actual prevision. There are also variants whose only occurrence is the feedback link from the output to the hidden layer (Figure 5.1b). This mechanism implements **memory** behaviors.

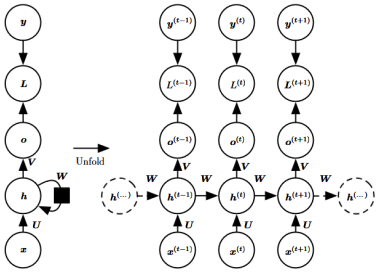
This architecture is designed to apply the same operations to all the data of the sequence. All inputs and outputs are dependent, because we apply the same operation on all data using past operations.

There are different architectures of RNN:

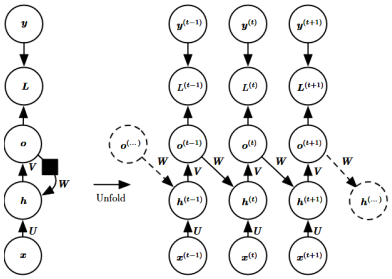
- RNNs that produces output for each time step and the recurrent connection from the previous hidden layer to the next hidden layer. (Figure 5.1a)
- RNN that produces output for each time step and the recurrent connection is from output to the next hidden layer. (Figure 5.1b)
- RNN that doesn't produce output for each time step, but we have the recurrent connection from output to the next hidden layer. At the end of the sequence we produce output. (Figure 5.1c)

The use of multiple layers in various parts of an RNN—input, hidden, and output layers—can enhance performance; however, there is currently no theoretical proof to support this.

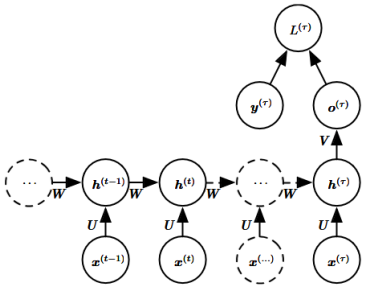
To train such models, we can apply a similar approach to the architectures discussed previously. However, some adjustments are necessary, as the information is derived not only from the input layers but also from past hidden layers.



(a) The computational graph of a recurrent network



(b) An RNN whose only recurrence is the feedback connection from the output to the hidden layer.



(c) Recurrent neural network with a single output at the end of the sequence

Computing the gradient of this loss function with respect to the parameters is an expensive operation. The gradient computation involves performing a forward propagation pass moving left to right through our illustration of the unrolled graph, followed by a backward propagation pass moving right to left through the graph. The runtime is $\mathcal{O}(\tau)$ and cannot be reduced by parallelization because the forward propagation graph is inherently sequential; each time step may be computed only after the previous one. States computed in the forward pass must be stored until they are reused during the backward pass, so the memory cost is also $\mathcal{O}(\tau)$. The back-propagation algorithm applied to the unrolled graph with $\mathcal{O}(\tau)$ cost is called **back-propagation through time** (BPTT).

For notation purposes let's use $o^{(t)} = \hat{y}^{(t)}$. Then consider:

$$h^{(t)} = \tanh(U \cdot x^{(t)} + W \cdot h^{(t-1)}) \ \wedge \ \hat{y}^{(t)} = softmax(V \cdot h^{(t)}) \tag{5.1}$$

And the cross entropy as the loss (i.e., error) function to minimize:

$$E(y, \hat{y}) = \sum_t E^{(t)}(y^{(t)}, \hat{y}^{(t)}) = -\sum_t y^{(t)} \cdot \log(\hat{y}^{(t)}) \tag{5.2}$$

The goal of BPTT is to calculate the gradients of the error with reference to U, V, W and then learn good values of these parameters using Stochastic Gradient Descent. Just like we sum up the errors, we also sum up the gradients at each time step for one training example:

$$\frac{\partial E}{\partial U} = \sum_t \frac{\partial E^{(t)}}{\partial U}, \ \wedge \ \frac{\partial E}{\partial V} = \sum_t \frac{\partial E^{(t)}}{\partial V}, \ \wedge \ \frac{\partial E}{\partial W} = \sum_t \frac{\partial E^{(t)}}{\partial W} \tag{5.3}$$

To calculate these gradients, we use the differentiation chain rule, which underpins the (traditional) back-propagation algorithm. While the computation of $\frac{\partial E}{\partial V}$ may be relatively straightforward depending on the RNN architecture, applying the chain rule to compute $\frac{\partial E}{\partial U}$ and $\frac{\partial E}{\partial W}$ is much more complex. This is because we must backpropagate the gradients from the output, through the network, and all the way back to $t = 0$.

The basic problem is that gradients propagated over many stages tend to either **vanish** (most of the time) or **explode** (rarely, but with much damage to the optimization). The difficulty with long-term dependencies arises from the exponentially smaller weights given to long-term interactions (involving the multiplication of many matrices) compared to short-term ones.

One might hope to avoid the problem by staying in a region of parameter space where gradients neither vanish nor explode. Unfortunately, for the RNN to store memories in a manner robust to small perturbations, it must enter a region of parameter space where gradients vanish. Specifically, whenever the model can represent long-term dependencies, the gradient of a long-term interaction has an exponentially smaller magnitude compared to the gradient of a short-term interaction. This does not mean it is impossible to learn long-term dependencies, but rather that it might take a very long time, as the signal associated with these dependencies is often overshadowed by the smallest fluctuations arising from short-term dependencies.

Since we need to compute a product of gradients at each time step, we may encounter one of the following issues:

- **Exploding gradients**: This occurs when all gradients are > 1. It can be mitigated by gradient clipping;
- **Vanishing gradients**: This arises when all gradients are < 1, leading to challenges in training. Solutions include using appropriate activation functions, weight initialization strategies, or modifying the network architecture, with the latter being the most effective approach for addressing vanishing gradients.

Solutions to the vanishing gradient problem are typically implemented as follows:

- **Activation functions**: ReLU is preferred over tanh or sigmoid, as it mitigates vanishing gradients more effectively. (Figure 5.2)

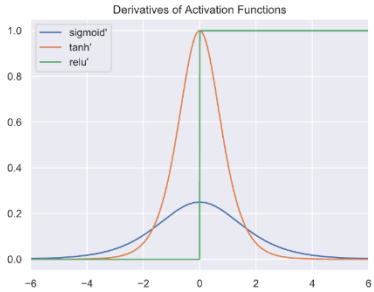


Figure 5.2: ReLU prevents f', to shrink gradients when $x > 0$

- **Initialization**: We can initialize weights as an identity matrix and biases as 0, though this approach is not universally effective.
- **Architecture**: Modifying the architecture by incorporating gates to regulate the flow of information (e.g., as seen in ResNet) often proves to be the most robust solution.

5.1 Gated RNNs

In standard RNNs Figure 5.3, repeating modules contain a simple computation node.

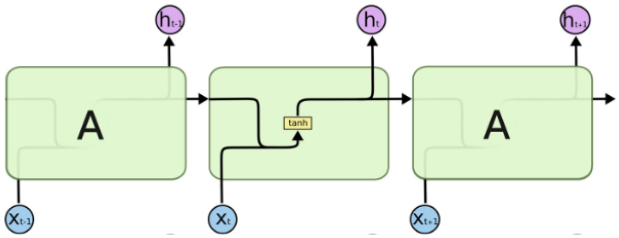


Figure 5.3: Standard RNN representation

The most effective sequence models used in practical applications are called **gated RNNs**. These include the **long short-term memory** (LSTM) and networks based on the **gated recurrent unit** (GRU). Gated RNNs are based on the idea of creating paths through time that have derivatives that neither vanish nor explode.

5.1.1 Long Short-Term Memory LSTM

The clever idea of introducing self-loops to produce paths where the gradient can flow for long durations is a core contribution of the initial **long short- term memory** (LSTM) model. A crucial addition has been to make the weight on this self-loop conditioned on the context, rather than fixed. By making the weight of this self-loop gated (controlled by another hidden unit), the time scale of integration can be changed dynamically.

In this case, we mean that even for an LSTM with fixed parameters, the time scale of integration can change based on the input sequence, because the time constants are output by the model itself.

The LSTM block diagram is illustrated in Figure 5.4. Instead of a unit that simply applies an element-wise nonlinearity to the affine transformation of inputs and recurrent units, LSTM recurrent networks have "LSTM cells" that have an internal recurrence (a self-loop), in addition to the outer recurrence of the RNN. Each cell has the same inputs and outputs as an ordinary recurrent network, but also has more parameters and a system of gating units that controls the flow of information.

LSTM cells are organized as follows:

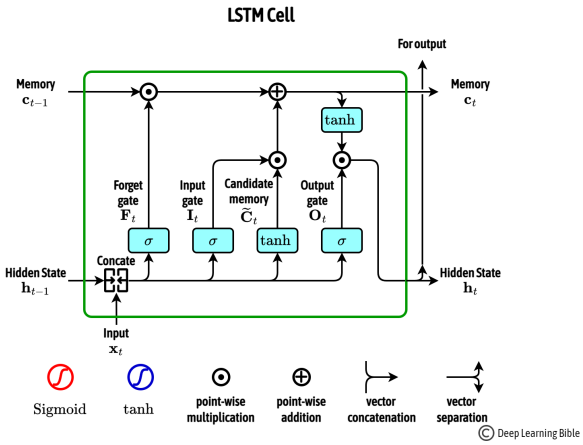


Figure 5.4: LSTM cell

- **Forget gate**: decides which information from long term memory be kept or discarded and this is done by multiplying the incoming long term memory by a forget vector generated by the current input and incoming short memory:

$$f_t = \sigma(W_f \cdot [h^{(t-1)}, x^{(t)}] + b_f)$$

- **Input gate**: decides what information will be stored in long term memory. It only works with the information from the current input and short term memory from the previous step. At this gate, it filters out the information from variables that are not useful. First, determine which entries in the cell state to update by computing $0 - 1$ sigmoid output:

$$i^{(t)} = \sigma(W_i \cdot [h^{(t-1)}, x^{(t)}] + b_i)$$

  Then determine what amount to add/subtract from these entries by computing a tanh output (valued -1 to 1) function of the input and hidden state:

$$\tilde{C}^{(t)} = \tanh(W_c \cdot [h^{(t-1)}, x^{(t)}] + b_c)$$

- **Output gate**: will take the current input, the previous short term memory and newly computed long term memory to produce new short term memory which will be passed on to the cell in the next time step. Hidden state is updated based on a *filtered* version of the cell state, scaled to -1 to 1 using tanh. Output gate computes a sigmoid function of the input and previous hidden state to determine which elements of the cell state to output:

$$h^{(t)} = \sigma(W_o \cdot [h^{(t-1)}, x^{(t)}] + b_o) \cdot \tanh C^{(t)}$$

The previous state is multiplied by the forget gate and then added to the fraction of the new candidate allowed by the input gate:

$$C^{(t)} = f^{(t)} \cdot C^{(t-1)} + i^{(t)} \cdot \tilde{C}^{(t)} \tag{5.4}$$

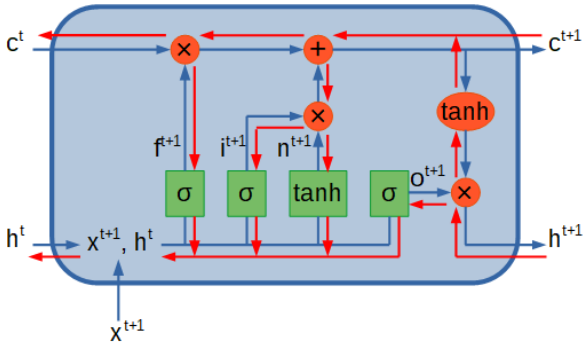


Figure 5.5: Backpropagation through time with uninterrupted gradient flow

5.1.2 GRU

To solve the problem faced by standard RNN, GRU incorporates the two gate operating mechanisms called **Update gate** and **Reset gate**. In particular, it uses fewer gates than LSTM by combining forget and input gates into update gates and eliminates cell state vector. This architecture has significant less number of parameters so it is faster than the other.

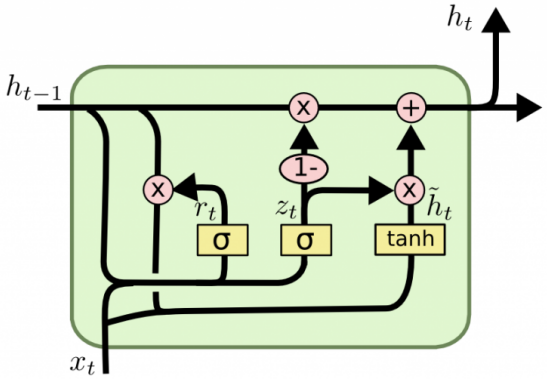


Figure 5.6: GRU cell

GRU cells are organized as follows:

- **Update Gate**: is responsible for determining the amount of previous information that needs to pass along the next state. This is really powerful because the model can decide to copy all the information from the past and eliminate the risk of vanishing gradient;
- **Reset Gate**: it stores relevant information from the past time step into new memory content. Then it multiplies the input vector and hidden state with their weights. Next, it calculates element-wise multiplication between the reset gate and previously hidden state multiple. After summing up the above steps the non-linear activation function is applied and the next sequence is generated.

Chapter 6

Generative Adversarial Networks

Many generative models are based on the idea of using a differentiable generator network. The model transforms samples of latent variables z to samples x or to distributions over samples x using a differentiable function $g(z; \theta^{(g)})$, which is typically represented by a neural network. This model class includes:

- **Variational auto-encoders**: which pair the generator net with an inference net;
- **Generative Adversarial Networks** (GAN): which pair the generator network with a discriminator network;
- Techniques that train generator networks in isolation.

Generator networks are essentially just parametrized computational procedures for generating samples, where the architecture provides the family of possible distributions to sample from and the parameters select a distribution from within that family.

The goal of this model is to generate something starting from random noise. In order to do this, we want to create a model that is capable **to learn to sample** the distribution of the training set. Usually, this models are unsupervised because they do not use a labels because we haven't any information on that.

Its possible to create model that are able to generate a new sample based on some condition, for examples generating a new instance starting from a given label. Moreover, this models can be use for *image to image translation*. In this case we solve task like given a black&white image and we want to generate the colored image. This feature makes these models very useful for performing computer vision tasks such as: increasing data, improving image, etc...

In order to build a generative model we need two main components:

- The first is to define an architecture that allows a new sample to be generated;
- The second is to define the correct loss function.

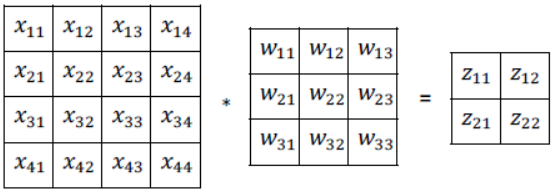
So we want to learn to sample, so $x \sim p_{data}$ should be $x_{gen} \sim p_{model}$, such manner that $p_{data} \equiv p_{model}$.

6.1 Generator architecture

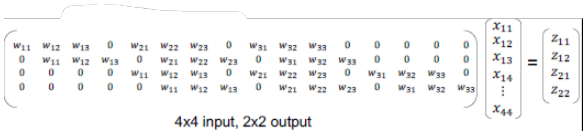
During this course, we have already explored a type of neural network capable of generating data: the *autoencoder*. Specifically, by isolating the decoder component and sampling from the latent space, we can generate novel examples.

Up to this point, we have explored architectures (e.g., CNNs) designed to reduce the size of images. Now, we aim to examine how these architectures can be adapted to up-sample from a noise distribution. We begin with a standard convolution operation, as illustrated in Figure 6.1a. This operation can be expressed in matrix form, as shown in Figure 6.1b.

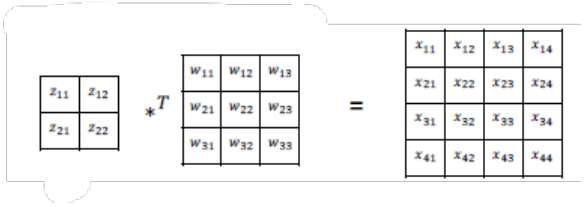
From this representation, we observe that the image size can be increased by multiplying the result by the transposed filter, as demonstrated in Figure 6.2a. It is important to note, however, that this operation **does not correspond to the inverse** of the original convolution. The figure 6.2b shows the result of this operation at the graphic level.



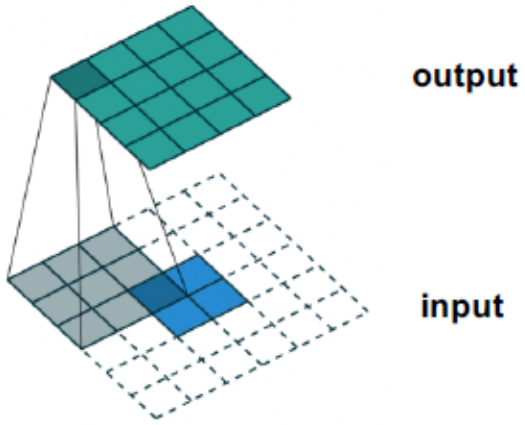
(a) Regular convolution stride 1, pad 0



(b) Matrix form



(a) Mathematically



(b) Visual representation

6.2 Generative Adversarial Networks

Generative Adversarial Networks (GANs) are built on a game-theoretic framework in which a generator network competes against an adversary. The generator network directly produces samples, $x = g(z; \theta^{(g)})$, where z is a noise vector, and $\theta^{(g)}$ represents the generator's parameters. Its adversary, the **discriminator network**, attempts to distinguish between real samples drawn from the training data and fake samples generated by the generator. The discriminator outputs a probability value, $d(x; \theta^{(d)})$, representing the likelihood that x is a real training example rather than a synthetic sample produced by the generator.

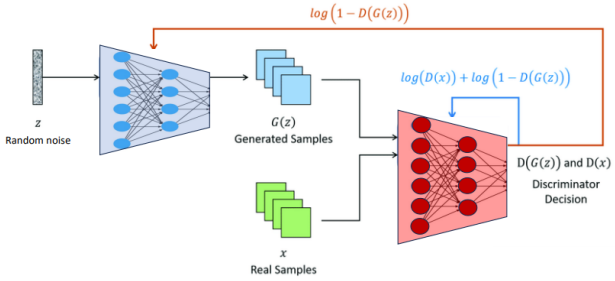


Figure 6.3: GAN architecture

6.2.1 Define the loss function

The simplest way to formulate learning in generative adversarial networks is as a zero-sum game, in which a function $v(\theta^{(g)}, \theta^{(d)})$ determines the payoff of the discriminator. The generator receives $-v(\theta^{(g)}, \theta^{(d)})$ as its own payoff. During learning, each player attempts to maximize its own payoff, so that at convergence:

$$g^* = \arg\min_g \max_d v(g, d) \tag{6.1}$$

With this approach we are training the generator and the discriminator jointly in a **minimax game**, where both need to get better and better to improve the generation quality.

The default choice for v is:

$$V(\theta^{(g)}, \theta^{(d)}) = \mathbb{E}_{x \sim p_{data}} \log d(x) + \mathbb{E}_{z \sim p_{model}} \log(1 - d(g(z))) \tag{6.2}$$

This drives the discriminator to attempt to learn to correctly classify samples as real or fake, in other words the discriminator wants to maximize:

$$d^* = \arg\max_d V(g, d)$$

Simultaneously, the generator attempts to fool the classifier into believing its samples are real:

$$G^* = \arg\min_G V(G, D)$$

In general, simultaneous gradient descent on two players' costs is not guaranteed to reach an equilibrium. Note that the equilibria for a minimax game are not local minima of v. Instead, they are points that are simultaneously minima for both players' costs. This means that they are *saddle points* of v that are local minima with respect to the first player's parameters and local maxima with respect to the second player's parameters.

The training process therefore consists of alternating between:

- Gradient ascent on discriminator:

$$D^* = \arg\max_D V(G, D) \tag{6.3}$$

- Gradient descent on generator (minimize log-probability of discriminator being right):

$$G^* = \arg\min_G V(G, D) = \arg\min_G \mathbb{E}_{z \sim p_{gen}} \log(1 - D(G(z))) = \arg\max_G \mathbb{E}_{z \sim p_{gen}} \log(D(G(z))) \tag{6.4}$$

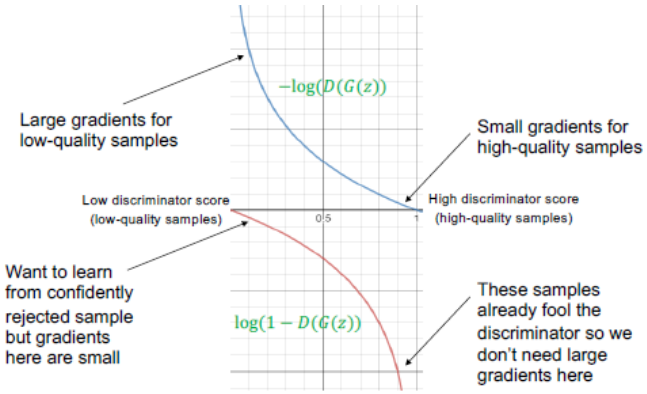


Figure 6.4: Loss function

In the original version the loss of the generator G learns fewer from initial image generated, but in this moment we want to learn more, the opposite thing happen for higher value of the discriminator and this isn't useful. So we rewrite the loss function using $1 - \log(D(G(z)))$ in order to learn more at the starting of the training.

The process for updating the generator increase $D(G(z))$ requires back-propagating through the composed generator-discriminator network i.e., the discriminator cannot be a black box. The generator is exposed to real data only via the output of the discriminator and its gradients.

GANs have different problems:

- **Unstability**: parameters can oscillate or diverge, generator loss does not correlate with sample quality and the behavior is sensitive to hyper-parameter selection;
- **Mode collapse**: the generator ends up modeling only a small subset of the training data and not the entire distribution.

Here some tips on GAN:

- **Conditional GAN**: It is possible to train conditional GANs that learn to sample from a distribution $p(x|y)$ rather than simply sampling from a marginal distribution $p(x)$;

- **Non-Saturating Generator Loss**: using the original generator loss when the discriminator can recognize each fake examples, this means that $D(G(X))$ is close to 0 so $\log(1 - D(G(X)))$ is close to 0 and the Generator can't learn. We can use Non-Saturating Generator Loss that solve this problem by maximizing $\log(D(G(X)))$. Another tip to solve this problem is to set threshold of real example lower, ex: 0.9 instead of 1 in discriminator;
- **Class-label discriminator**: If we have a dataset of C classes we can use a discriminator that try to classify data between C+1 classes, where the C+1 class is the fake one. Empirically improves generation quality because the discriminator focus on features that are important for humans;
- **Virtual batch norm**: Batch normalization typically normalizes data within each mini-batch during training, but this can cause issues in GANs because can add instability and also artifacts. We can use Virtual batch norm which use a reference batch used for compute statistics and use them to normalized the actual mini-batch;
- **Mini-batch discrimination**: using a discriminator that use just an example cannot detect mode collapse, we can solve by adding to any images some features derived from entire mini-batch.

6.3 Evaluating Generative Models

Generative modeling is different from normal model because changes in preprocessing, even very small and subtle ones, are completely unacceptable. Any change to the input data changes the distribution to be captured and fundamentally alters the task.

Because being able to generate realistic samples from the data distribution is one of the goals of a generative model, practitioners often evaluate generative models by visually inspecting the samples. Unfortunately, it is possible for a very poor probabilistic model to produce very good samples. Imagine a generative model trained on images of dogs and cats that simply learns to reproduce the training images of dogs. Such a model has clearly overfit, because it does not produces images that were not in the training set, but it has also underfit, because it assigns no probability to the training images of cats. Yet a human observer would judge each individual image of a dog to be high quality. In more realistic settings, a generative model trained on data with tens of thousands of modes may ignore a small number of modes, and a human observer would not easily be able to inspect or remember enough images to detect the missing variation.

Since the visual quality of samples is not a reliable guide, we often also evaluate the log-likelihood that the model assigns to the test data, when this is computationally feasible. Unfortunately, in some cases the likelihood seems not to measure any attribute of the model that we really care about.

The fact that there are many different uses of generative models the choice of metric must match the intended use of the model.

We pass from discriminator to update a generator because we don't know which loss put in output to generator.

Chapter 7

Federated Learning

The standard setting in Machine Learning (ML) considers a centralized dataset processed in a tightly integrated system, but in the real world data is often decentralized across many parties. One challenge is that collecting these data at a single point is not always feasible for various reasons. Transmitting the data might be prohibitively expensive, such as when dealing with large volumes of data, or it may be constrained by the limited bandwidth or power capacity of wireless devices.

Another challenge is that the data might be believed too sensitive. In recent years, there has been increasing public awareness and stricter regulations concerning data privacy. Additionally, maintaining control over data can offer a competitive edge in both business and research contexts.

One possible solution could be to create a separate model for each data source, utilizing the data available locally. However, this approach has significant drawbacks. The amount of data at each source may be too limited, leading to overfitting or decisions that lack statistical significance. Moreover, the local dataset may fail to accurately represent the overall target distribution, further compromising the model's effectiveness.

Federated Learning (FL) aims to collaboratively train a ML model while keeping the data decentralized. The methodology for implementing federated learning is as follows:

1. The server initialize the model weights and send a copy to all the parties;
2. Each party make an update using its local dataset;
3. Parties share local updates for aggregation;
4. Server aggregates updates and send back to parties;
5. Parties update their copy of the model and iterate.

We would like the final model to be as good as the centralized solution (ideally), or at least better than what each party can learn on its own.

Note 26. *The assumption is that starting from the weights we cannot obtain the data, otherwise we could have just send the data.*

More broadly, we can categorize federated learning (FL) into different classifications. The first classification differentiates between:

- **Cross-device FL**: this scenario involves a large number of participants, each possessing a small dataset. Additionally, it is important to account for the fact that some participants may not always be reachable, and some may have malicious intentions, posing further challenges.
- **Cross-silo**: in this scenario we have a limited number of parties with medium large dataset. Parties are always available and are typically honest.

Another type of classification is:

- **Server orchestrated FL**: where we have a server that performs global coordination and aggregation. The server is the bottleneck and a single point of failure.
- **Device-to-device communication**: peer to peer communication between the parties. This approach can scale better.

7.1 Distributed Learning

A different training paradigm is **distributed learning**. In this approach, unlike federated learning (FL), the data is initially centralized and then distributed later to accelerate model training. The key distinction from FL lies in how the data is distributed: in distributed learning, the data is typically distributed uniformly at random across workers under controlled conditions.

In contrast, FL involves data that is naturally distributed and generated locally. This data is often not independent and identically distributed (non-i.i.d.) and is usually imbalanced. FL also introduces additional challenges that span various fields of computer science, such as enforcing privacy constraints, addressing the potentially limited reliability and availability of participants, ensuring robustness against malicious actors, and more.

7.2 Baseline algorithm: FedAvg

Let's start by introducing some notation. Let's consider a set of K parts (clients). Each part k holds a data set $\mathcal{D}_k$ composed of n_k instances. Let $\mathcal{D} = \mathcal{D}_1 \cup \cdots \cup \mathcal{D}_K$ be the total dataset and $n = \sum_k n_k$ the total number of instance.

In this situation, we want to solve the problems of the form:

$$\min_{\theta \in \mathbb{R}^p} F(\theta; \mathcal{D}) \tag{7.1}$$

where:

$$F(\theta; \mathcal{D}) = \sum_{k=1}^{K} \frac{n_k}{n} F_k(\theta; \mathcal{D}_k) \quad \text{and} \quad F_k(\theta; \mathcal{D}_k) = \sum_{d \in \mathcal{D}_k} f(\theta; d) \tag{7.2}$$

where $\theta \in \mathbb{R}^p$ are model parameters such as neural network weights.

Algorithm: FedAvg (server-side)
Parameters: client sampling rate ρ

Initialize θ
for each round $t = 0, 1, \ldots$ **do**
  $S_t \leftarrow$ random set of $m = \lceil \rho K \rceil$ clients
  for each client $k \in S_t$ in parallel **do**
    $\theta_k \leftarrow \text{ClientUpdate}(k, \theta)$
  end for
  $\theta \leftarrow \sum_{k \in S_t} \frac{n_k}{n} \theta_k$
end for

Algorithm: ClientUpdate(k, θ)
Parameters: batch size B, number of local steps L, learning rate η

for each local step $l = 1, \ldots, L$ **do**
  $B \leftarrow$ mini-batch of b examples from $\mathcal{D}_k$
  $\theta \leftarrow \theta - \frac{\eta}{B} \sum_{d \in B} \nabla \ell(\theta; d)$
end for
send θ to server

Notes:

- For $L = 1$ and $\rho = 1$, it is equivalent to classic parallel SGD: updates are aggregated, and the model is synchronized at each step.
- For $L > 1$, each client performs multiple local SGD steps before communicating.

FedAvg with $L > 1$ allows to reduce the number of communication rounds, which is often the bottleneck in FL. It empirically achieves better generalization than parallel SGD with large mini-batch. Convergence to the optimal model can be guaranteed for i.i.d. data, but issues arise in strongly non-i.i.d. cases.

Learning from non-i.i.d. data is difficult/slow because each party wants the model to go in a particular direction. If data distributions are very different, learning a single model which performs well for all parties may require a very large number of parameters.

Another direction to deal with non-i.i.d. data is thus to lift the requirement that the learned model should be the same for all parties. Instead, we can allow each party k to learn a (potentially simpler) personalized model θ_k but design the objective so as to enforce some kind of collaboration.

Several strategies have been developed to manage the combination of this last idea, including:

- A first approach propose to regularize personalized models to their mean:

$$F(\theta_1, \ldots, \theta_k; \mathcal{D}) = \frac{1}{K} \sum_{k=1}^{K} F_k(\theta_k; \mathcal{D}) + \frac{\lambda}{2 \cdot K} \sum_{k=1}^{K} \left\| \theta_k - \frac{1}{K} \sum_{i=i}^{K} \theta_i \right\| \tag{7.3}$$

- Another approach, inspired by **meta-learning**, propose to learn a global model which easily adapts to each party:

$$F(\theta; \mathcal{D}) = \frac{1}{K} \sum_{k=1}^{K} F_k(\theta - \alpha \cdot \nabla F_k(\theta); \mathcal{D}) \tag{7.4}$$

We can remove outliers using median at the place of mean.

Chapter 8

Transformers

Transformer were initially targeted at natural language processing (NLP) problems, where the network input is a series of high-dimensional embeddings representing words or word fragments. This architecture was designed with the goal of processing this text into a representation suitable for downstream tasks.

We can also encounter different problems such as:

- The encoded input can be very large. Assuming a 1024-d embedding and body of text that have 100s/1000s of words, a fully connected neural network is impractical because we have $100 \times 1024 = 102400$ as input length;
- Each input is of different length and Feed Forward Neural Networks are impracticable;
- Statistics are similar at every position;
- The language is fundamentally ambiguous.

Most of these problems are derived by NLP problems.

8.1 Dot-product self attention

Considering the issues just presented, we want to design a network that is able to process text and can:

1. Use parameter sharing to cope with long input passages of differing lengths;
2. Contain connections between word representations that depend on the words themselves.

The transformer acquires both properties by using **Dot-Product Self-Attention**. This mechanism is shown in Figure 8.1.

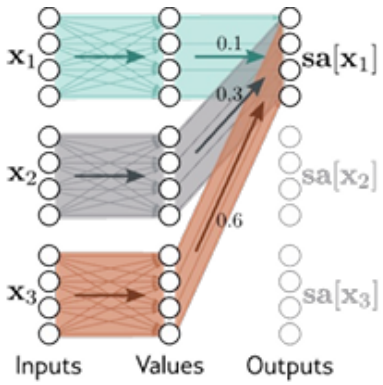


Figure 8.1: Dot-Product Self-Attention mechanism

While a standard NN layer $f[x]$ takes a $D \times 1$ input x and applies a linear transformation followed by a non-linear activation function $a[\cdot]$:

$$f[x] = a[\beta + \Omega x]$$

where β contains the biases and Ω contains weights. A self-attention block $sa[\cdot]$ takes N inputs x_n, each of dimension $D \times 1$, and returns N output vectors of the same size. In the context of NLP, each x_n will represent a word or a word fragment of a sentence.

Let's go into more detail. First, a set of **values** are computed for each input:

$$v_n = \beta_v + \Omega_v \cdot x_n \tag{8.1}$$

where β_v contains the biases and Ω_v contains the weights. Then the n-th output $sa[x_n]$ is a weighted sum of all the values v_n:

$$sa[x_n] = \sum_{m=1}^{N} a[x_m, x_n] \cdot v_m \tag{8.2}$$

The scalar weight $a[x_m, x_n]$ is the **attention** that input x_n pays to input x_m. The N weights $a[\cdot, x_n]$ are non-negative and sum to one.

Self-attention can be understood as a mechanism that routes the values in different proportions to construct each output.

To compute the values, the same weights Ω_v $(D \times D)$ and biases β_v $(D \times 1)$ are applied to each input x_n $(D \times 1)$:

$$v_n = \beta_v + \Omega_v \cdot x_n \tag{8.3}$$

This computation scales linearly with the sequence length N, so it requires less parameters than a fully connected neural network connecting all DN inputs to the DN outputs.

The value computation can thus be viewed as a sparse matrix multiplication (see Figure 8.2) with shared parameters.

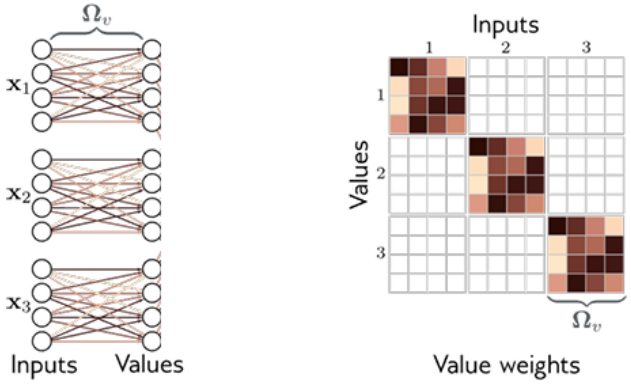


Figure 8.2: Values computation in self-attention using shared parameters

The attention weights $a[x_m, x_n]$ combine the values from different inputs. They are also sparse (Figure 8.3), since there is only one weight for each ordered pair of inputs (x_m, x_n)regardless of the size of these inputs.

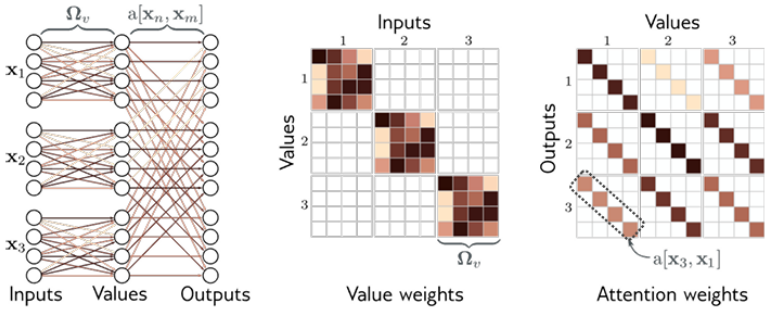


Figure 8.3: Attention

The number of attention weights has a quadratic dependence on the sequence length N but it is independent of the length D of each input x_n.

Through this process, you get the same output as that from two-chained linear transformations:

- The value vectors are computed independently from each input;
- and these vectors are linearly combined by the attention weights.

However, the overall self-attention is non-linear because the attention weights are themselves non-linear functions of the input. This is an example of a **hypernetwork**, where one network branch computes the weights of another branch.

To compute the *attention*, we apply two more linear transformations to the inputs:

$$q_n = \beta_q + \Omega_q \cdot x_n \quad \wedge \quad k_n = \beta_k + \Omega_k \cdot x_n \tag{8.4}$$

where q_n and k_n are referred to as the **queries** and the **keys**.

We then compute dot products between the queries and the keys followed by a softmax:

$$a[x_m, x_n] = softmax_m[k_*^T, q_n] = \frac{\exp[k_*^T, q_n]}{\sum_{m'=1}^{N} \exp[k_*^T, q_n]} \tag{8.5}$$

so, for each x_n they are positive and sum to one and these is the method to apply non linearity on attention.

The names *queries* and *keys* have the following interpretation:

- The dot-product operation returns a measure of similarity between its inputs, so the weights $a[x_*, x_n]$ depend on the relative similarities between each query and the keys;
- The softmax function means that we can think of the key vectors as *competing* with one another to contribute to the final result.

The queries and keys must have the same dimensions.

However, these can differ from the dimension of the values, which is usually the same size as the input, so that the representation does not change size.

This mechanism fulfills the initial requirements:

1. There is a single shared set of parameters $\phi = \{\beta_v, \Omega_v, \beta_q, \Omega_q, \beta_k, \Omega_k\}$. This is independent of the number of inputs N, so the networks can be applied to difference sequence lengths;
2. There are connections between the inputs (words), and the strength of these connections depends on the inputs themselves via the attention weights.

This can be written in a compact form if the N inputs x_n form the columns of the $D \times N$ matrix X. The values, queries, and keys can be computed as:

$$V[X] = \beta_v \cdot 1^T + \Omega_v \cdot X$$
$$Q[X] = \beta_q \cdot 1^T + \Omega_q \cdot X$$
$$K[X] = \beta_k \cdot 1^T + \Omega_k \cdot X$$

The self-attention computation is then:

$$sa[X] = V[X] \cdot softmax[K[X]^T, Q[X]] \tag{8.6}$$

where softmax is independently applied on the columns of its input.

Since the dot-product in the attention calculation can result in large magnitudes, it pushes the arguments of the softmax function into a region where the largest value dominates entirely. This causes small changes to the softmax inputs to have minimal impact on the output, making the model harder to train.

To mitigate this, the dot products are scaled by the square root of the size of the query D_q and key D_k, which corresponds to the number of rows in Ω_q and Ω_k as the query and key dimensions must be the same.

$$sa[X] = V \cdot softmax\left[\frac{K^T \cdot Q}{\sqrt{D_q}}\right] \tag{8.7}$$

8.2 Positional encoding

The self-attention mechanism discards important information, in fact, the computation is the same regardless of the order of the inputs. However, the order is important when the inputs correspond to the words in a sentence. There are two main approaches to incorporating position information:

- **Absolute position embeddings**;
- **Relative position embeddings**.

8.2.1 Absolute position embeddings

This method consists of adding a Π matrix to the X input. The purpose of this matrix is to encode positional information. Each column of Π is unique and contains information about the absolute position in the input sequence.

The matrix that is usually used for this purpose can be handcrafted or learned in the model training phase. Typically, this matrix is added to the model's input. Alternatively, since the outputs of the self-attention blocks have the same dimensions as the input, the matrix can be added after each layer.

Note 27. *Sometimes it is added to X in the computation of the queries and keys but not to the values.*

8.2.2 Relative position embeddings

In this case, we are considering a situation where the input to a self-attention mechanism may be an entire sentence, many sentences, or just a fragment of a sentence, and the absolute position of a word is much less important than the relative position between two inputs. Of course, this can be recovered if the system knows the absolute position of both, but relative position embeddings encode this information directly. Each element of the attention matrix corresponds to a particular offset between query position a and key position b.

Relative position embeddings learn a parameter $\pi_{a,b}$, for each offset and use this to modify the attention matrix by adding these values, multiplying by them, or using them to alter the attention matrix in some other way.

8.3 Multiple heads self attention

In the most used architectures, multiple self-attention mechanisms are used in parallel, and this is known as **Multi-Head Self-Attention**. If we now consider the fact of having H self-attention blocks, we have that the different sets of values, keys and queries are calculated as:

$$V_h[X] = \beta_{vh} \cdot 1^T + \Omega_{vh} \cdot X$$
$$Q_h[X] = \beta_{qh} \cdot 1^T + \Omega_{qh} \cdot X$$
$$K_h[X] = \beta_{kh} \cdot 1^T + \Omega_{kh} \cdot X$$

Then, the $h-$th self-attention mechanism or **head** can be written as:

$$sa_h[X] = V_h \cdot softmax\left[\frac{K_h^T \cdot Q_h}{\sqrt{D_q}}\right] \tag{8.8}$$

where we have different parameters for each head.

Typically, if the dimension of the inputs x_m is D and there are H heads, the values, queries, and keys will all be of size D/H as this allows for an efficient implementation. The outputs of these self-attention mechanisms are then vertically concatenated, and another linear transform Ω_c is applied to combine them. An example with $H = 2$ is shown in Figure 8.4. The output of the multi-head self-attention mechanism is then:

$$Mhsa[X] = \Omega_c[sa_1[X]^T, sa_2[X]^T, \ldots, sa_H[X]^T] \tag{8.9}$$

This last transformation is useful because it allows to bring the weights of the different heads in the same range. In addition, if necessary, this combination can be used to limit a head that provides wrong data in output by setting it to zero.

Multiple heads seem to be necessary to make the transformer work well. It has been speculated that they make the self-attention network more robust to bad initializations.

8.4 Transformer Layers

All the components we have defined so far serve as a basis for defining the **Transformer Layer**. This is the basic building block of the transformer architecture. The structure of a Transformer Layer is shown in Figure 8.5.

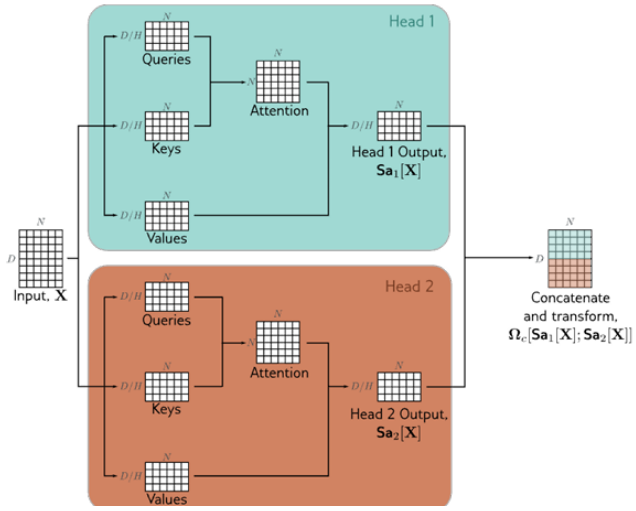


Figure 8.4: Multi-Head Self-Attention example with $H = 2$

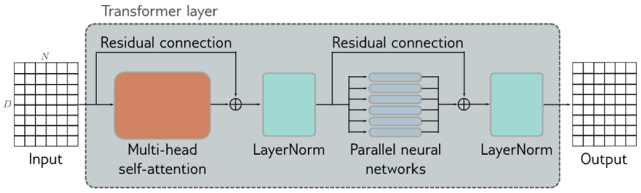


Figure 8.5: Transformer Layers

Transformer Layer consists of a multi-head self-attention unit, which allows the word representations to interact with each other, followed by a fully connected network $mlp[x_*]$ that operates separately on each word. Both units are residual networks.

In addition, it is typical to add a **LayerNorm** operation after both the Self-Attention and Fully Connected Networks.

Architecture that are now used in practice are more complex in the sense that they have more than one layer of self-attention and fully connected networks.

8.5 NLP pipeline

A typical NLP pipeline begins with a **tokenizer**, which splits text into a vocabulary of smaller constituent units (tokens) that the subsequent network can process. These tokens often represent words, but several challenges arise:

- Inevitably, some words (e.g., names) will not be present in the vocabulary;
- Handling punctuation can be unclear, yet crucial. For example, if a sentence ends with a question mark, this information must be encoded;
- The vocabulary would require separate tokens for different forms of the same word (e.g., variations with different suffixes), with no mechanism to clarify that these variations are related.

Each of these tokens is mapped to a learned embedding. These embeddings are then passed through a series of transformer layers. We will now examine each of these stages in detail.

In practice, a compromise between letters and full words is employed. The final vocabulary includes both common words and word fragments, which can be combined to represent larger and less frequent words. This vocabulary is typically generated using a sub-word tokenizer, such as byte pair encoding (BPE), which greedily merges frequently occurring substrings based on their frequency.

Each token in the vocabulary $\mathcal{V}$ is consistently mapped to a corresponding word embedding, ensuring that the same token always maps to the same embedding.

To accomplish this, the N input tokens are represented in the matrix T, with dimensions $|V| \times N$. Each column of T corresponds to a token, where the n-th column represents the n-th token as a one-hot vector of size $|V| \times 1$.

The embeddings for the entire vocabulary are stored in a matrix Ω_e, with dimensions $D \times |V|$. The input embeddings are computed as $X = \Omega_e \cdot T$, where Ω_e is learned like any other parameter of the network.

A typical embedding size D is 1024, and a typical vocabulary size $|V|$ is 30,000. As a result, even before the main network processes the data, the matrix Ω_e contains a large number of parameters to learn.

Finally, the embedding matrix X, which represents the text, is passed through a series of transformer layers, forming what is known as a transformer model.

There are three types of transformer models:

1. **Encoders**: An encoder transforms text embeddings into representations that support a variety of tasks. **BERT** (Bidirectional Encoder Representations from Transformers) is an example of an encoder model with a vocabulary of 30,000 tokens. Input tokens are mapped to 1,024-dimensional word embeddings and processed through 24 transformer layers, each containing a self-attention mechanism with 16 heads. Each head computes queries, keys, and values with a dimension of 64. Encoder models like BERT employ transfer learning. During pretraining, the transformer's parameters are learned via self-supervision on a large text corpus. The pretraining objective is to capture general statistical knowledge of language through tasks such as predicting missing words in sentences. The learned text representations can then be fine-tuned for specific NLP tasks.

2. **Decoders**: A decoder generates new tokens to extend an input text sequence. For example, **GPT-3** constructs an autoregressive language model, where the joint probability of a sequence of tokens $(t_1, t_2, \ldots, t_N)$ is factored as:

$$P(t_1, t_2, \ldots, t_N) = P(t_1) \cdot \prod_{n=2}^{N} P(t_n \mid t_1, \ldots, t_{n-1}) \tag{8.10}$$

   This autoregressive approach defines a probability distribution over text sequences, enabling the generation of new, coherent text. Given an input sequence, the model predicts a probability distribution over possible next tokens. The selected token is appended to the sequence and fed back into the model, repeating the process iteratively to generate text.

3. **Encoder-Decoders**: Encoder-decoder models are designed for sequence-to-sequence tasks, such as **machine translation**, where text in one language is transformed into another. These models encode the input sequence into a latent representation and decode it into the target sequence.

Chapter 9

Self-Supervised Learning

The models trained from large-scale of data, such as the ImageNet dataset, are widely used as the pre-trained models and fine-tuned for other tasks for two main reasons:

- The parameters learned from large-scale diverse datasets provide a good starting point, therefore, networks training on other tasks can converge faster;
- The network trained on large-scale datasets already learned the hierarchy features which can help to reduce over-fitting problem during the training of other tasks, especially when datasets of other tasks are small, or training labels are scarce.

With the sophisticated architectures and large-scale datasets, the performance of DNNs keeps breaking the state-of-the-arts for many tasks. However, collection and annotation of large-scale datasets are time-consuming and expensive.

To avoid time-consuming and expensive data annotations, many **self-supervised** methods were proposed to learn visual features from large-scale unlabeled images or videos without using any human annotations.

A popular solution is to propose various pretext tasks for networks to solve. The networks can be trained by learning objective functions of the pretext tasks and in the while the features are learned through this process. Thanks to the pretext tasks, the data are labeled automatically.

The model created using this method can be used as a pre-trained model and applied to the supervised downstream tasks to improve the performance.

Various pretext tasks have been proposed for self-supervised learning including colorizing gray scale images, image inpainting, playing image jigsaw puzzle, etc. The pretext tasks share two common properties:

- Visual features of images or videos need to be captured by CNNs to solve the pretext tasks;
- The supervisory signal is generated from the data itself (self-supervision) by leveraging its structure.

During the self-supervised training phase, a predefined pretext task is designed for CNNs to solve, and the pseudo labels for the pretext task are automatically generated based on some attributes of data.

Then the CNN is trained to learn objective functions of the pretext task. As with training on a labeled dataset, when trained with pretext tasks, the shallower blocks of CNN focus on the low-level general features such as corners, edges, and textures, while the deeper blocks focus on the high-level task-specific features such as objects, scenes, and object parts. Therefore, CNNs trained with pretext tasks can learn kernels that capture low-level features and high-level features that are helpful for other downstream tasks.

After the self-supervised training is finished, the learned visual features can be further transferred to downstream tasks as pre-trained models to improve performance and overcome over-fitting.

The main advantage of self-supervised learning methods is that they can be easily scaled to large-scale datasets with very low cost.

We now present the terminology that will be used during this section:

- **Human-annotated label**: labels of data that are manually annotated by human workers.
- **Pretext Task**: pre-designed tasks for networks to solve, and visual features are learned by learning objective functions of pretext tasks. The pretext tasks can be predictive tasks, generative tasks, contrasting tasks, or a combination of them. The supervision signal for pretext tasks is generated from the data itself based on its structure.

- **Pseudo label**: The labels used in pretext task is referred as Pseudo labels which are generated based on the structure of data for pretext tasks.
- **Downstream Task**: CV applications that can be used to evaluate the quality of features learned by self-supervised learning. These applications can greatly benefit from the pre-trained models when training data are scarce. In general, human-annotated labels are needed to solve the downstream tasks. However, in some applications, the downstream task can be the same as the pretext task without using any human-annotated labels.
- **Supervised Learning**: refers to learning methods using data with fine-grained human annotated labels to train networks.
- **Semi-supervised Learning**: refers to learning methods using a small amount of labeled data in conjunction with a large amount of unlabeled data.
- **Weakly-supervised Learning**: refers to learning methods to learn with coarse-grained labels or inaccurate labels. The cost of obtaining weak supervision labels is generally much cheaper than fine-grained labels for supervised methods.
- **Unsupervised Learning**: refers to learning methods without using any human-annotated labels.
- **Self-supervised Learning**: refers to learning methods in which CNNs are explicitly trained with supervisory signals that are generated from the data itself (self-supervision) by leveraging its structure.

We can compare the performance of image feature training by evaluating the classificator on the same dataset.

Note 28. *Self supervised learning add robustness to model.*

9.1 Learning visual features from pretexts tasks

To relieve the burden of large-scale dataset annotation, a pretext task is generally designed for networks to solve while pseudo labels for the pretext task are automatically generated based on data attributes. Effective pretext tasks are those ensuring that semantic features are learned through the process of accomplishing the pretext tasks.

As we have seen before, there are different types of pretext task that can be used to train a neural network with self supervised learning. We can classify them into four categories as shown in Figure 9.1.

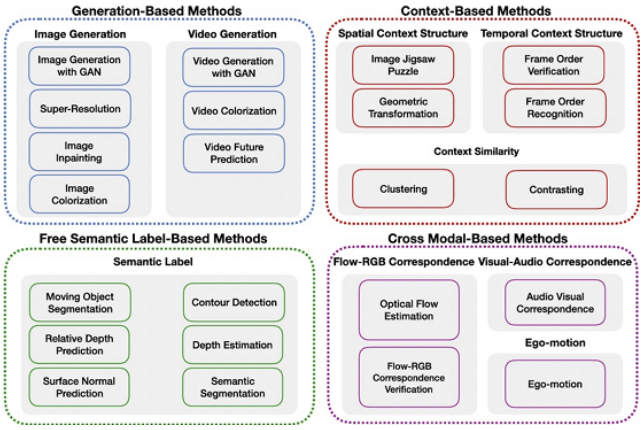


Figure 9.1: Commonly used pretext tasks.

9.1.1 Generation-based pretext tasks

This type of methods learn visual features by solving pretext tasks that involve image or video generation.

- **Image Generation**: Visual features are learned through the process of image generation tasks. This type of methods includes image colorization, image super resolution, image inpainting, image generation with Generative Adversarial Networks (GANs).

- **Video Generation**: Visual features are learned through the process of video generation tasks. This type of methods includes video generation with GANs, and video prediction.

For GAN we train the network and we use only the discriminator and we take features from last layers.

Generation-based self-supervised methods for learning image features involve the process of generating images. For these tasks, pseudo training labels P usually are the images themselves and no human annotated labels are needed during training, therefore, these methods belong to self-supervised learning methods.

The pioneer work about the image generation-based methods is the Autoencoder which learns to compress an image into a low-dimension vector and then is uncompressed into an image that is close to the original image with a bunch of layers.

9.1.2 Context-based pretext tasks

The design of context-based pretext tasks mainly employs the context features of images or videos such as context similarity, spatial structure, temporal structure, etc. as the supervision signal.

- **Context Similarity**: Pretext tasks are designed based on the context similarity between image patches. This type of methods includes image clustering-based methods, and graph constraint-based methods. There are two ways of utilizing context similarity as supervision signals for self-supervised learning: formulating it as a **predictive task** or a **contrastive task**.

  For both methods, the data are first clustered into different groups under the assumption that data from the same group have high context similarity, while data from different groups have low context similarity.

  - **Predictive Task**. The predictive tasks involve training networks to predict the group ID of the data, usually with a cross entropy loss. In this context, the clustering methods are mainly employed as a tool to cluster image data.

    After the clustering, several clusters are obtained while the image within one cluster have a smaller distance in feature space and images from different clusters have a larger distance in feature space.

    The smaller the distance in feature space, the more similar the image in the appearance in the RGB space.

    Then a CNN can be trained to classify the data by using the cluster assignment as the pseudo class label. To accomplish this task, the CNN needs to learn the invariance within one class and the variance among different classes. Therefore, the CNN learns semantic meaning of images.
  - **Contrastive Task**. The contrastive tasks involve training networks to directly minimize feature distances from the same group and maximize feature distances from different groups, usually with a triplet loss or a contrastive loss.

    The general idea of the contrastive SSL is to train networks to maximum agreement of different views of same scene while minimizing agreement of views from different scenes.

    The recent state-of-the-art method is SimCLR which learns features by contrasting images after a composition of data augmentations. The positive pairs are constructed by sampling two images after applying different augmentation techniques for the same image, while negative pairs include two different images.

- **Spatial Context Structure**: Pretext tasks used to train CNNs are based on the spatial relations among image patches. This type of methods includes image jigsaw puzzle, context prediction, and geometric transformation recognition, etc.

  The pretext task can be to predict the relative positions of two patches from same image, or to recognize the order of a shuffled sequence of patches from same image. The context of full images can also be used as a supervision signal to design pretext tasks such as to recognize the rotating angles of the whole images. To accomplish these pretext tasks, CNNs need to learn spatial context information such as the shape of the objects and the relative positions of different parts of an object.

  To limit the number of permutations, usually, Hamming distance is employed to choose only a subset of permutations among all the permutations, i.e., those with a relative large Hamming distance. Only the selected permutations are used to train CNN to recognize the permutation of shuffled image patches.

- **Temporal Context Structure**: The temporal order from videos is used as supervision signal. The CNN is trained to verify whether the input frame sequence in correct order, or to recognize the order of the frame sequence.

Features are learned by CNN through the process of solving the pretext tasks designed based on attributes of the context of images.

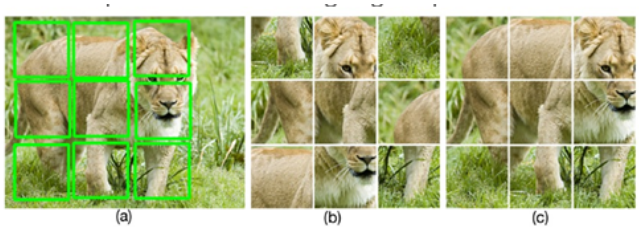


Figure 9.2: Spatial context structure-based pretext tasks.

9.1.3 Free semantic label-based pretext tasks

This type of pretext tasks train networks with automatically generated semantic labels. Generally, the free semantic labels such as segmentation masks, depth images, optical flows, and surface normal images can be rendered by game engine or generated by hard-code methods.

Since these semantic labels are automatically generated, the methods using the synthetic datasets or using them in conjunction with a large unlabeled image or video datasets are considered as self-supervised learning methods.

Note 29. *Strictly speaking, the methods based on data generated by game engines do/should not belong to the self-supervised learning methods since human intervention is needed during the data generation process. However, some recent work treat them as self-supervised learning methods.*

Game engines can generate realistic images with accurate pixel-level labels with very low cost. However, due to the domain gap between synthetic and real-world images, the CNNs purely trained on synthetic images cannot be directly applied to real-world images. To utilize synthetic datasets for self-supervised feature learning, the domain gap needs to be explicitly bridged.

To overcome the problem, unsupervised feature space domain adaptation methods based on adversarial learning have been proposed. The CNN predicts surface normal, depth, and instance contour for the synthetic images and a discriminator network D is employed to minimize the difference of feature space domains between real-world and synthetic data.

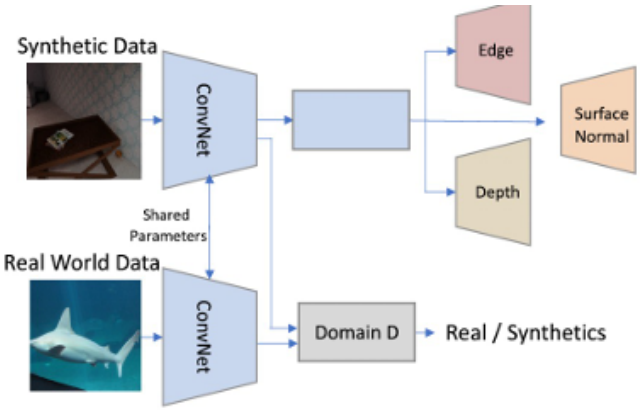


Figure 9.3: Free semantic labels-based pretext tasks.

Applying hard-code programs is another way to automatically generate semantic labels such as salience, foreground masks, contours, depth for images and videos. With these methods, very large-scale datasets with generated semantic labels can be used for self-supervised feature learning. This type of methods generally has two steps:

1. Label generation by employing hard-code programs on images or videos to obtain labels;
2. Train CNNs with the generated labels.

No matter what kind of labels used to train CNNs, the general idea of this type of methods is to distill knowledge from the hard-code detector.

One drawback is that the semantic labels generated by hard-code detector usually are very noisy which need to be specifically coped with.

9.1.4 Cross modal-based pretext tasks

This type of pretext tasks trains CNNs to verify whether two different channels of input data are corresponding to each other such as Visual-Audio Correspondence Verification, RGB-Flow Correspondence Verification, Contrasting (i.e., maximizing agreement between differently augmented views of the same data example), and egomotion.

Egomotion is defined as the 3D motion of a camera within an environment. In the field of computer vision, egomotion refers to estimating a camera's motion relative to a rigid scene. An example of egomotion estimation would be estimating a car's moving position relative to lines on the road or street signs being observed from the car itself. The estimation of egomotion is important in autonomous robot navigation applications.

Cross modal-based learning methods usually learn features from the correspondence of multiple data streams.

In addition to rich temporal and spatial information in videos, optical flow sequence can be generated to specifically indicate the motion in videos, and the difference of frames can be computed with negligible time and space-time complexity to indicate the boundary of the moving objects.

Based on the type of data used, these methods fall into three groups:

- Methods that learn features by using the RGB and optical flow correspondence;
- Methods that learn features by utilizing the video and audio correspondence
- Ego-motion that learn by utilizing the correspondence between egocentric video and egomotorsensor signals.

Usually, the network is trained to recognize if the two kinds of input data are corresponding to each other, or is trained to learn the transformation between different modalities.

9.2 Commonly used downstream tasks for evaluation

To evaluate the quality of the learned image or video features by self-supervised methods, the learned parameters by SSL are employed as pre-trained models and then fine-tuned on downstream tasks such as image classification, semantic segmentation, object detection, and action recognition etc.

The performance of the transfer learning on these high-level vision tasks demonstrates the generalizability of the learned features.

If CNNs trained in SSL scenarios can learn general features, then the pre-trained models can be used as a good starting point for other vision tasks that require capturing similar features from images or videos.

In addition to the quantitative evaluations of the learned features, there are also some qualitative visualization methods to evaluate the quality of SSL features. Three methods are often used for this purpose:

1. **Kernel visualization**: Qualitatively visualize the kernels of the first convolution layer learned with the pretext tasks and compare the kernels from supervised models. The similarity of the kernels learned by supervised and self-supervised models are compared to indicate the effectiveness of self supervised methods.

2. **Feature Map Visualization**: Feature maps are visualized to show the attention of networks. Larger activation represents the neural network pays more attention to the corresponding region in the image. Feature maps are usually qualitatively visualized and compared with that of supervised models.

3. **Nearest Neighbor Retrieval**: In general, images with similar appearance usually are closer in the feature space. The nearest neighbor method is used to find the top K nearest neighbors from the feature space of the features learned by the self-supervised learned model.

Chapter 10

Introduction to Explainable AI

The term **Explainable AI** (XAI) has been defined in various ways throughout the literature. In this chapter, we provide an overview of the most common definitions and concepts. Additionally, we discuss the significance of XAI and the motivation driving its development.

To begin, let us clarify what XAI is not. **Explainable AI does not imply causality.** Providing an explanation does not necessarily mean the model has identified a cause-and-effect relationship. AI models typically identify patterns in data rather than causal links. Therefore, even if a feature is identified as important to a prediction, it does not imply that the feature caused the observed outcome.

Explanations in AI can be influenced by human biases and the subjective choice of explanation methods. Different explainability techniques may produce varying explanations for the same prediction, potentially leading to inconsistencies. Furthermore, explanations can be affected by hidden biases in the training data or the model's underlying logic.

It is important to note that an explanation is not the same as rigorous proof or scientific validation. Explaining a model's behavior does not equate to formally verifying its reasoning. A model's reasoning may sound convincing but still be flawed, as explanations can sometimes rely on spurious correlations or logically unsound premises, even if they appear coherent.

Despite these limitations, explainability is crucial for building trust in AI systems. We can define XAI as methods and models that make the behavior and predictions of machine learning (ML) systems understandable to humans. This is important for several reasons:

- **Trust**: Users are more likely to trust understandable models, particularly in high-stakes applications;
- **Debugging**: Enables users to inspect the patterns discovered by AI models and identify potential issues;
- **Fairness**: Helps ensure that the learned relationships are not influenced by human biases;
- **Informativeness**: Facilitates the transfer of knowledge from ML models to humans;
- **Legal compliance**: Addresses requirements set by regulations such as GDPR and the AI Act.

10.1 Taxonomy of XAI Methods

There is no single way to categorize XAI methods. Here, we introduce a taxonomy based on four dimensions:

10.1.1 Framework

- **Interpretable models**: Models that are inherently interpretable, such as linear regression or decision trees;
- **Explainability algorithms**: Techniques designed to explain the predictions of existing black-box models.

10.1.2 Scope

- **Global**: Explains the model's behavior across the entire dataset;
- **Local**: Explains the model's behavior for a specific prediction.

10.1.3 Applicability

- **Model-specific**: Explanations tailored to a specific model type;
- **Model-agnostic**: Explanations that are applicable to any model type.

10.1.4 Produced Results

- **Feature importance**: Highlights the contribution of each feature to the model's predictions;
- **Counterfactual explanations**: Explains how input changes would alter the output;
- **Concepts**: Provides explanations in terms of human-friendly concepts or abstractions rather than input features;
- Additional techniques as applicable.

10.2 SHAP

SHAP (SHapley Additive exPlanations) is a unified approach to explain the outputs of machine learning models. It is based on cooperative game theory, which studies interactions between rational agents. In this context, the "agents" are the features of the input data, and the goal is to predict the model's output.

SHAP values are derived from the Shapley value, a concept from cooperative game theory. SHAP computes all possible coalitions of features and evaluates the contribution of each feature to the prediction using a utility function. This *utility function* represents the total value the coalition achieves by working together. The Shapley values provide a fair distribution of this value among the features in the coalition.

10.3 LIME

LIME (Local Interpretable Model-agnostic Explanations) is a model-agnostic technique for explaining predictions made by machine learning models. LIME works by approximating the decision boundary of the original model locally, around a specific prediction. It creates a simpler and interpretable surrogate model that approximates the behavior of the complex model in the neighborhood of the given prediction.